Fraudulent Claim Detection

Problem Statement

Global Insure, a leading insurance company, processes thousands of claims annually. However, a significant percentage of these claims turn out to be fraudulent, resulting in considerable financial losses. The company's current process for identifying fraudulent claims involves manual inspections, which is time-consuming and inefficient. Fraudulent claims are often detected too late in the process, after the company has already paid out significant amounts. Global Insure wants to improve its fraud detection process using data-driven insights to classify claims as fraudulent or legitimate early in the approval process. This would minimise financial losses and optimise the overall claims handling process.

Business Objective

Global Insure wants to build a model to classify insurance claims as either fraudulent or legitimate based on historical claim details and customer profiles. By using features like claim amounts, customer profiles and claim types, the company aims to predict which claims are likely to be fraudulent before they are approved.

Based on this assignment, we have to answer the following questions:

- How can we analyse historical claim data to detect patterns that indicate fraudulent claims?
- Which features are most predictive of fraudulent behaviour?
- Can we predict the likelihood of fraud for an incoming claim, based on past data?
- What insights can be drawn from the model that can help in improving the fraud detection process?

Assignment Tasks

We need to perform the following steps for successfully completing this assignment:

1. Data Preparation
1. Data Cleaning
2. Train Validation Split 70-30
3. EDA on Training Data
4. EDA on Validation Data (optional)
5. Feature Engineering
6. Model Building
7. Predicting and Model Evaluation

Executive Summary

Global Insure faces substantial financial losses due to fraudulent claims, compounded by an inefficient manual detection process. This project aimed to develop a data-driven model to accurately classify insurance claims as fraudulent or legitimate early in the approval process. Through comprehensive data preparation, cleaning, exploratory data analysis, and feature engineering, two machine learning models—**Logistic Regression and Random Forest**—were built and evaluated. The Random Forest model, after hyperparameter tuning, demonstrated superior performance on the validation dataset (Accuracy: 82.00%, F1-Score: 0.675), making it a suitable candidate for deployment to mitigate financial losses and optimize claims handling.

Objectives

The primary objectives of this assignment were to:

- **Analyze Historical Claim Data:** Detect patterns indicating fraudulent claims.
- **Identify Predictive Features:** Determine which features are most predictive of fraudulent behavior.
- **Predict Fraud Likelihood:** Develop a model to predict the likelihood of fraud for incoming claims based on past data.
- **Derive Actionable Insights:** Extract insights from the model to improve the fraud detection process.

To achieve these, the following steps were performed:

- **Data Preparation**
- **Data Cleaning**
- **Train-Validation Split (70-30)**
- **EDA on Training Data**
- **EDA on Validation Data (optional)**
- **Feature Engineering**
- **Model Building (Logistic Regression and Random Forest)**
- **Prediction and Model Evaluation**

Data Overview

The dataset, insurance_claims.csv, initially contained 1000 rows and 40 columns. It includes various details about customers, their policies, incident specifics, claim amounts, and a target variable fraud_reported (Y/N).

Key observations during the data overview included:

- **Data Types:** A mix of numerical (int64, float64) and categorical (object) features.
- **Missing Values:** The _c39 column was entirely null (1000 missing values) and authorities_contacted had 91 missing values.
- **Inconsistent Values:** Several categorical columns (collision_type, property_damage, police_report_available) contained '?' values.
- **Redundant Columns:** Features like policy_number, policy_bind_date, insured_zip, and incident_location were identified as having high cardinality/unique values, indicating them as potential identifiers rather than predictive features.
- **Illogical Values:** A negative value was found in umbrella_limit.

Methodology

The project followed a structured machine learning workflow:

- **Data Preparation:** Loaded the dataset and inspected its basic structure, dimensions, and data types.
- **Data Cleaning:** Dropped the _c39 column due to 100% missing values.
- Imputed missing values in authorities_contacted with 'Other'.
- Replaced '?' in collision_type, property_damage, and police_report_available with 'Unknown'.
- Corrected the illogical negative value in umbrella_limit to 0 (mode).
- Removed high-cardinality columns (policy_number, policy_bind_date, insured_zip, incident_location) as they offered limited predictive power.
- Converted incident_date to datetime objects.
- **Train-Validation Split:** The dataset was split into 70% training and 30% validation sets, stratified by the fraud_reported target variable to maintain class distribution.

- **Exploratory Data Analysis (EDA):**
    - **Univariate Analysis:** Histograms were used to visualize the distribution of numerical features.
    - **Correlation Analysis:** A heatmap identified correlations among numerical features.
    - **Class Balance:** The target variable's distribution was examined, confirming a class imbalance.
    - **Bivariate Analysis:** Relationships between categorical features and the target, and numerical features and the target, were explored using bar plots and box plots respectively.
- **Feature Engineering:**
    - **Resampling:** RandomOverSampler was applied to the training data to address class imbalance.
    - **Feature Creation:** incident_month was extracted from incident_date, and auto_age was calculated (max_year - auto_year).
    - **Redundant Column Handling:** age (due to high correlation with months_as_customer), total_claim_amount (as it's a sum of other claims), and incident_date (after month extraction) were dropped.
    - **Categorical Combination**: incident_hour_of_the_day was binned into incident_period_of_the_day categories (Night, Morning, Afternoon, Evening).
    - **Dummy Variable Creation:** Categorical features were one-hot encoded using get_dummies with drop_first=True.
    - **Feature Scaling:** Numerical features were scaled using StandardScaler.
- **Model Building and Evaluation:**
    - **Logistic Regression**: Recursive Feature Elimination with Cross-Validation (RFECV) was used for feature selection. A statsmodels.api.GLM model was built, and multicollinearity was assessed using Variance Inflation Factor (VIF). Features with high VIF were dropped. Model performance was evaluated using accuracy, confusion matrix, sensitivity, specificity, precision, recall, and F1-score. Optimal cutoff was determined by analyzing ROC, accuracy-sensitivity-specificity, and precision-recall curves.
    - **Random Forest**: A baseline RandomForestClassifier was built. Feature importances were extracted, and only the most important features (importance >= 0.02) were retained. The model was re-trained, and cross-

validation was performed to check for overfitting. Hyperparameter tuning was conducted using GridSearchCV to optimize model parameters.

Key Findings

1. **Data Quality:** Initial data contained significant missing values and inconsistent entries, which were addressed during cleaning.

2. **Class Imbalance:** The target variable fraud_reported showed a notable imbalance, with legitimate claims outweighing fraudulent ones. This was managed using oversampling.

3. **Feature Importance:** The Random Forest model identified incident_severity_Minor Damage, vehicle_claim, insured_hobbies_chess, incident_severity_Total Loss, capital-loss, insured_hobbies_cross-fit, and witnesses as the most crucial features for predicting fraud.

4. **Model Performance (Training Data):** Logistic Regression: Achieved an accuracy of approximately 89.94% and an F1-Score of 0.901 on the resampled training data.

5. **Random Forest (Untuned):** Initially showed 100% accuracy on training data, indicating overfitting. Cross-validation revealed a mean accuracy of 92.13%, suggesting better generalization than initial training metrics.

6. **Random Forest (Tuned):** After hyperparameter tuning, the training accuracy slightly reduced to 86.24% and F1-Score to 0.864, indicating a more robust model.

7. **Model Performance (Validation Data):**
    - Logistic Regression: Accuracy: 80.33%, Sensitivity: 62.16%, Specificity: 86.28%, Precision: 59.74%, Recall: 62.16%, F1-Score: 0.609.
    - Random Forest (Tuned): Accuracy: 82.00%, Sensitivity: 75.68%, Specificity: 84.07%, Precision: 60.87%, Recall: 75.68%, F1-Score: 0.675.

8. **Model Comparison:** The tuned Random Forest model generally outperformed Logistic Regression on the validation set, particularly in terms of Sensitivity (correctly identifying fraudulent claims) and F1-Score, which are critical metrics in fraud detection.

Recommendations

1. **Deploy Tuned Random Forest Model:** Given its superior performance on validation data, the tuned Random Forest model is recommended for implementation in Global Insure's fraud detection system.

2. **Focus on Key Predictive Features:** Leverage insights from the highly important features (incident severity, vehicle claim, specific hobbies, capital-loss, witnesses) to refine fraud investigation protocols and potentially design new data collection strategies.

3. **Implement Continuous Monitoring:** Establish a system for continuous monitoring of the model's performance in a real-world environment. Regular retraining with new data will ensure its effectiveness over time.

4. **Explore Advanced Techniques:** Consider investigating more sophisticated machine learning techniques such as Gradient Boosting (e.g., XGBoost, LightGBM) or exploring different ensemble methods to potentially achieve even higher predictive accuracy and robustness.

5. **Cost-Benefit Analysis of Cutoff:** Conduct a detailed cost-benefit analysis to determine the optimal probability cutoff that balances the costs of false positives (investigating legitimate claims) and false negatives (missing fraudulent claims).

Assumptions:

1. The provided insurance_claims.csv dataset is a fair and representative sample of Global Insure's claim data.

2. The fraud_reported labels are accurate indicators of true fraud instances.

3. The imputation of '?' values to 'Unknown' in categorical columns did not introduce significant bias or distort underlying patterns.

4. The removal of high-cardinality features and highly correlated features did not discard any non-obvious, crucial predictive information.

5. The characteristics of future incoming claims will be similar to the historical data used for training and validation.

Data Dictionary

The insurance claims data has 40 Columns and 1000 Rows. Following data dictionary provides the description for each column present in dataset:

1. Data Preparation

In this step, read the dataset provided in CSV format and look at basic statistics of the data, including preview of data, dimension of data, column descriptions and data types.

1.0 Import Libraries

```
# Supress unnecessary warnings
import warnings
warnings.filterwarnings("ignore")
```

```
# Import necessary libraries
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

1.1 Load the Data

```
# Load the dataset
df = pd.read_csv("insurance_claims.csv")
```

```
# Check at the first few entries
df.head()
```

{"type":"dataframe","variable_name":"df"}

```
# Inspect the shape of the dataset
df.shape
```

```
(1000, 40)
```

```
# Inspect the features in the dataset
df.info()
```

2. Data Cleaning [10 marks]

2.1 Handle null values [2 marks]

2.1.1 *Examine the columns to determine if any value or column needs to be treated [1 Mark]*

```
# Check the number of missing values in each column
df.isnull().sum().sort_values(ascending=False)
```

```
df.isnull().sum().sort_values(ascending=False)/ df.shape[0] * 100
```

```
_c39                          100.0
authorities_contacted           9.1
months_as_customer              0.0
age                             0.0
```

Authorities_contacted has 9.1% (91) of null values, _c39 has 100% (1000) null values.

***2.1.2** Handle rows containing null values [1 Mark]*

```
df.authorities_contacted.value_counts()
```

```
authorities_contacted
Police       292
Fire         223
Other        198
Ambulance    196
Name: count, dtype: int64
```

```
# Handle the rows containing null values
df['authorities_contacted'] =
df['authorities_contacted'].fillna(value='Other', axis=0)
```

2.2 Identify and handle redundant values and columns [5 marks]

***2.2.1** Examine the columns to determine if any value or column needs to be treated [2 Mark]*

```
# Write code to display all the columns with their unique values and counts
and check for redundant values
for col in df.columns:
    print(f"Column: {col}")
    print(df[col].value_counts())
    print("\n" + "-" * 30 + "\n")
```

***2.2.2** Identify and drop any columns that are completely empty [1 Mark]*

```
# Identify and drop any columns that are completely empty
# Dropping the column _c39 as it has 100% null values and is not useful
df.drop('_c39', axis=1, inplace=True)
```

```
df.shape
```

```
(1000, 39)
```

***2.2.3** Identify and drop rows where features have illogical or invalid values, such as negative values for features that should only have positive values [1 Mark]*

```
# Identify and drop rows where features have illogical or invalid values,
such as negative values for features that should only have positive values
df.umbrella_limit.value_counts()
```

```
# Umbrella_limit has a negative value which should either have zero or
positive value
# Find the index of the negative value. There is only one.
df.umbrella_limit.argmin()
```

```
np.int64(290)
```

```
df.umbrella_limit[290]
```

```
np.int64(-1000000)
```

```
#This can be safely imputed with mode which is 0 for this feature
df.umbrella_limit[290] = df.umbrella_limit.mode()
```

```
df.umbrella_limit[290]
```

```
np.int64(0)
```

```
# Identify and drop rows where features have illogical or invalid values,
such as ?
inv_value_columns = {}
for column in df.select_dtypes(include='object').columns :
    total = np.sum(df[column] == '?')
    inv_value_columns[column] = total
columns_with_invalid_values = pd.DataFrame.from_dict(inv_value_columns,
orient='index')
columns_with_invalid_values.columns = ['Invalid_count']
print(columns_with_invalid_values)
```

```
                         Invalid_count
policy_bind_date                     0
policy_state                         0
policy_csl                           0
insured_sex                          0
insured_education_level              0
insured_occupation                   0
insured_hobbies                      0
insured_relationship                 0
incident_date                        0
incident_type                        0
collision_type                     178
incident_severity                    0
authorities_contacted                0
incident_state                       0
incident_city                        0
incident_location                    0
property_damage                    360
police_report_available            343
auto_make                            0
auto_model                           0
fraud_reported                       0
```

```
#As there are significant unknown(?) values, it is best to replace the '?'
with 'Unknown' rather than deleting rows
df['collision_type'] = df.collision_type.replace({'?': 'Unknown'})
df['property_damage'] = df.property_damage.replace({'?':'Unknown'})
df['police_report_available'] =
df.police_report_available.replace({'?':'Unknown'})
```

```
display(df.collision_type.value_counts())
display(df.property_damage.value_counts())
display(df.police_report_available.value_counts())
```

```
collision_type
Rear Collision     292
Side Collision     276
Front Collision    254
Unknown            178
Name: count, dtype: int64

property_damage
Unknown    360
NO         338
YES        302
Name: count, dtype: int64

police_report_available
Unknown    343
NO         343
YES        314
Name: count, dtype: int64
```

2.2.4 *Identify and remove columns where a large proportion of the values are unique or near-unique, as these columns are likely to be identifiers or have very limited predictive power [1 Mark]*

```
# Identify and remove columns that are likely to be identifiers or have very limited predictive power
for column in df.columns :
    if df[column].nunique() > 950 :
        print(f"{column} : {df[column].nunique()}")
```

```
policy_number : 1000
policy_bind_date : 951
policy_annual_premium : 991
insured_zip : 995
incident_location : 1000
```

```
# Features policy_number, policy_bind_date, policy_annual_premium, insured_zip, incident_location has unique values
# Drop the above features excluding policy_annual_premium as they have limited predictive power
df.drop(['policy_number','policy_bind_date','insured_zip','incident_location'], axis=1, inplace=True)
```

```
# Check the dataset
df.head()
```

{"type":"dataframe","variable_name":"df"}

```
df.shape
```

```
(1000, 35)
```

2.3 Fix Data Types [3 marks]

Carefully examine the dataset and identify columns that contain date or time information but are not stored as the appropriate data type. Convert these columns to the correct datetime data type to enable proper analysis and manipulation of temporal information.

```
# Fix the data types of the columns with incorrect data types
df.dtypes

# Feature incident_date has incorrect data type
# Convert incident_date to datetime format
df['incident_date'] = pd.to_datetime(df.incident_date)

# Check the features of the data again
df.info()
```

3. Train-Validation Split [5 marks]

3.1 Import required libraries

```
# Import train-test-split
from sklearn.model_selection import train_test_split
```

3.2 Define feature and target variables [2 Marks]

```
# Put all the feature variables in X
X = df.drop('fraud_reported', axis=1)

# Put the target variable in y
y = df['fraud_reported']

# Convert target variable to numerical (0 and 1)
y = y.map({'Y': 1, 'N': 0})
```

3.3 Split the data [3 Marks]

```
# Split the dataset into 70% train and 30% validation and use stratification
on the target variable
X_train, X_test , y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42, stratify=y)

# Reset index for all train and test sets
X_train = X_train.reset_index(drop=True)
X_test = X_test.reset_index(drop=True)
y_train = y_train.reset_index(drop=True)
y_test = y_test.reset_index(drop=True)

#Check the shape of Train and Test sets
print('Train Shape: ', X_train.shape, y_train.shape)
print('Test Shape: ', X_test.shape, y_test.shape)
```

```
Train Shape:  (700, 34) (700,)
Test Shape:  (300, 34) (300,)
```

4. EDA on training data [20 marks]

4.1 Perform univariate analysis [5 marks]

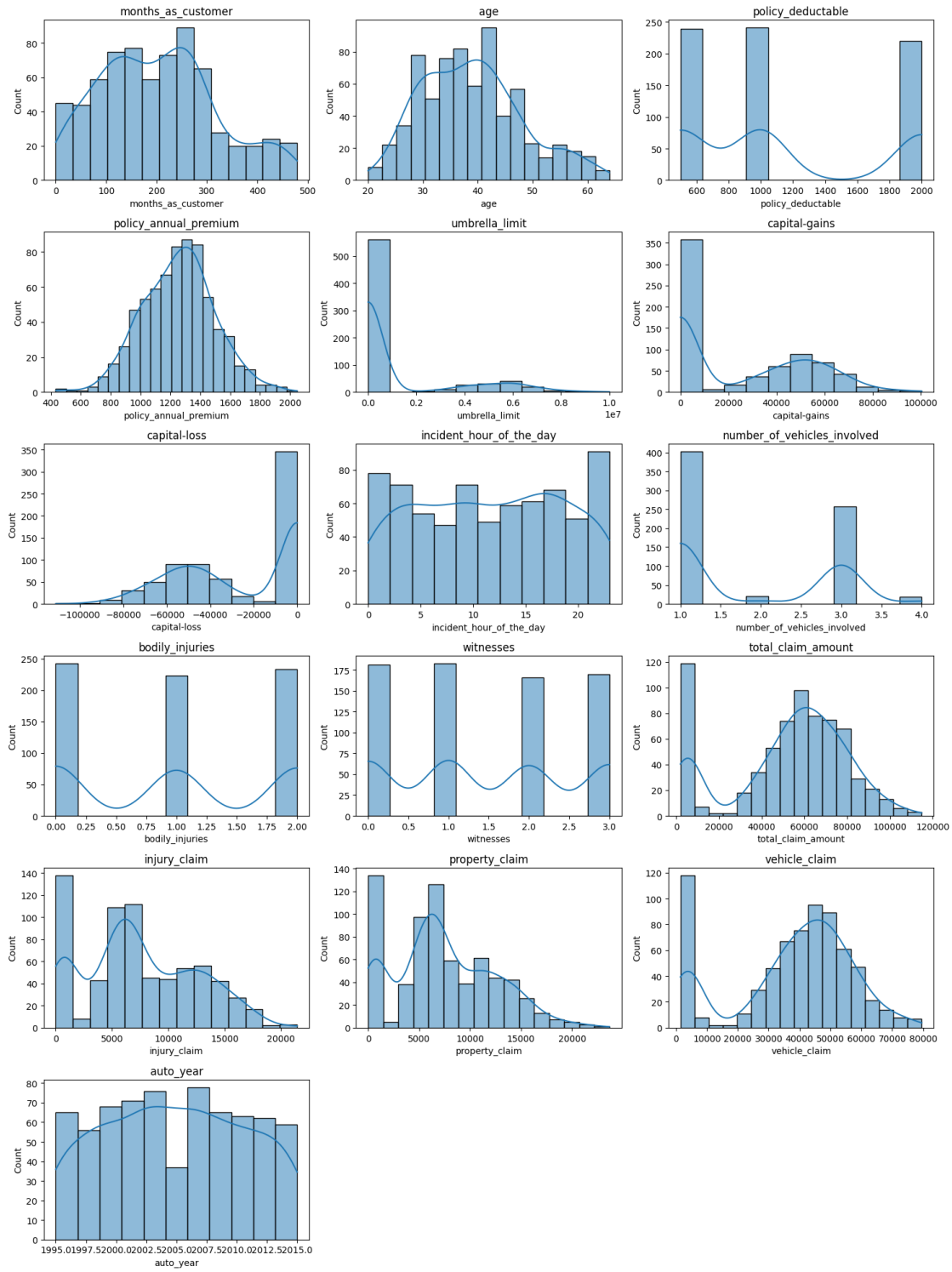
4.1.1 *Identify and select numerical columns from training data for univariate analysis [1 Mark]*

```
# Select numerical columns
numerical_cols_train = X_train.select_dtypes(include=np.number).columns

numerical_cols_train
```

```
Index(['months_as_customer', 'age', 'policy_deductable',
       'policy_annual_premium', 'umbrella_limit', 'capital-gains',
       'capital-loss', 'incident_hour_of_the_day',
       'number_of_vehicles_involved', 'bodily_injuries', 'witnesses',
       'total_claim_amount', 'injury_claim', 'property_claim',
'vehicle_claim',
       'auto_year'],
      dtype='object')
```

4.1.2 *Visualise the distribution of selected numerical features using appropriate plots to understand their characteristics [4 Marks]*

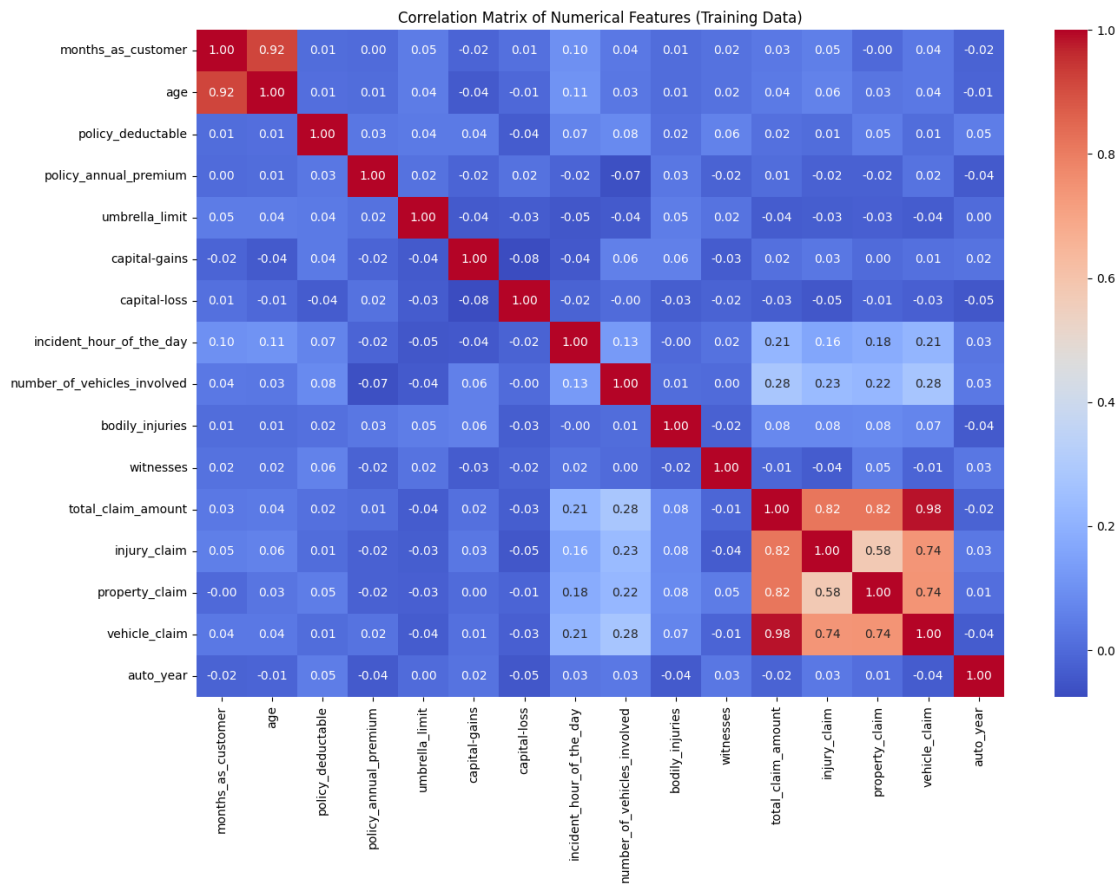
```
# Plot all the numerical columns to understand their distribution
plt.figure(figsize=(15, 20))


for i, col in enumerate(numerical_cols_train):
    plt.subplot(6, 3, i + 1)
    sns.histplot(X_train[col], kde=True)
    plt.title(col)


plt.tight_layout()
plt.show()
```



4.2 Perform correlation analysis [3 Marks]

Investigate the relationships between numerical features to identify potential multicollinearity or dependencies. Visualise the correlation structure using an appropriate method to gain insights into feature relationships.

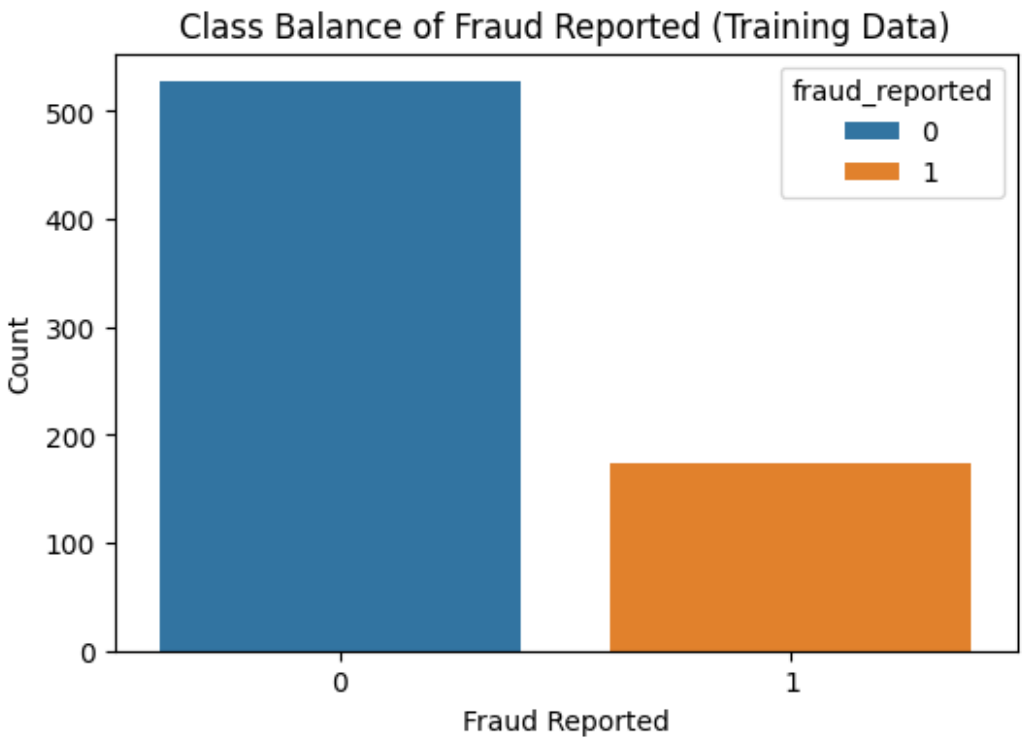
```python
# Create correlation matrix for numerical columns
corr_matrix_train = X_train[numerical_cols_train].corr()

# Plot Heatmap of the correlation matrix
plt.figure(figsize=(15, 10))
sns.heatmap(corr_matrix_train, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features (Training Data)')
plt.show()
```



4.3 Check class balance [2 Marks]

Examine the distribution of the target variable to identify potential class imbalances using visualisation for better understanding.

```python
# Plot a bar chart to check class balance
plt.figure(figsize=(6, 4))
sns.countplot(x=y_train, hue=y_train)
plt.title('Class Balance of Fraud Reported (Training Data)')
plt.xlabel('Fraud Reported')
plt.ylabel('Count')
plt.show()
```



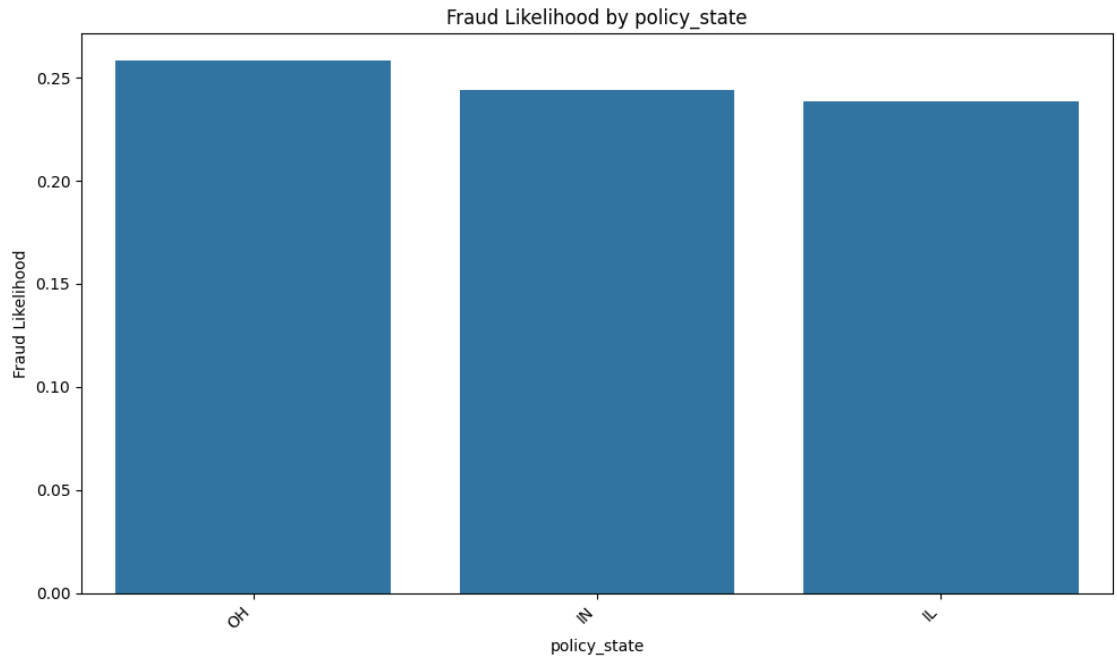
4.4 Perform bivariate analysis [10 Marks]

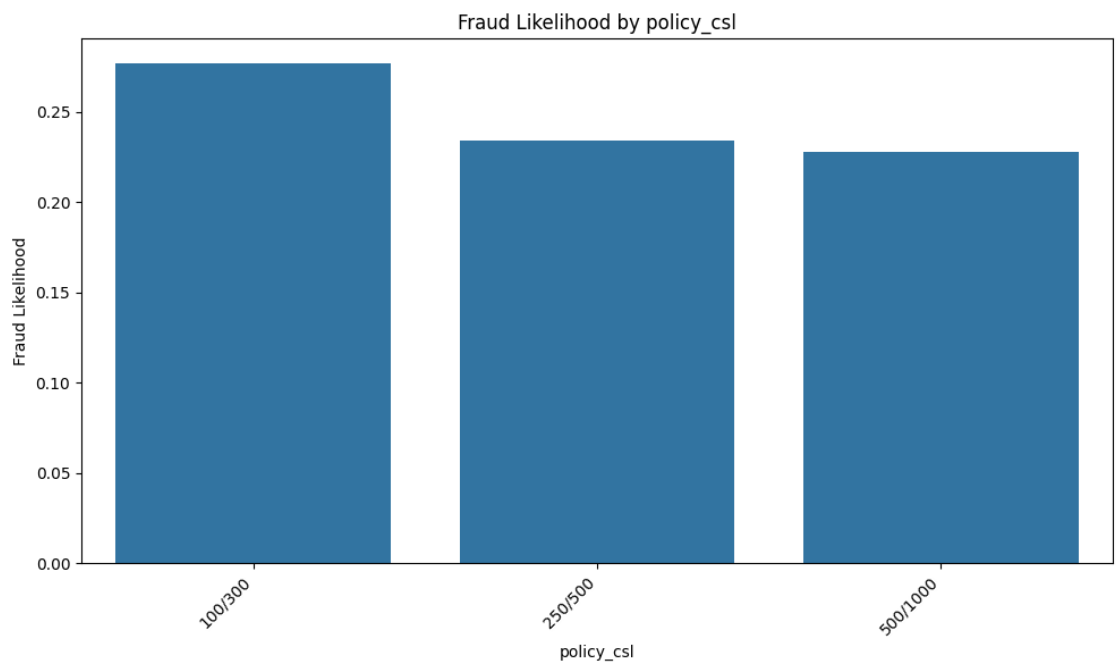
4.4.1 Target likelihood analysis for categorical variables. [5 Marks]

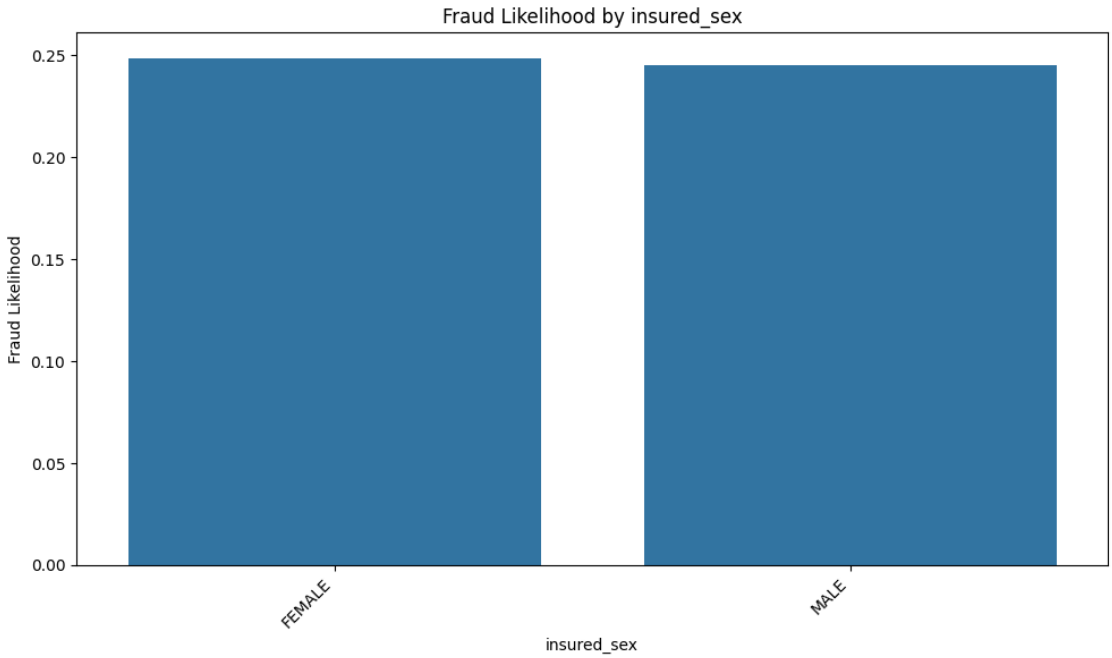
Investigate the relationships between categorical features and the target variable by analysing the target event likelihood (for the `'Y'` event) for each level of every relevant categorical feature. Through this analysis, identify categorical features that do not contribute much in explaining the variation in the target variable.

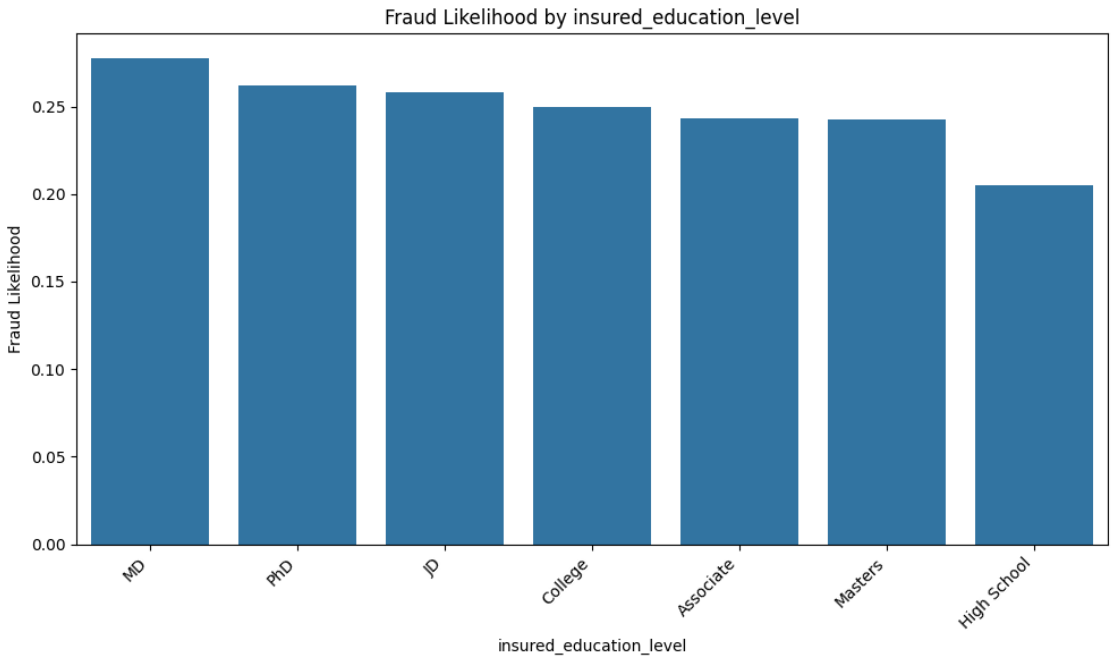
```
# Write a function to calculate and analyse the target variable likelihood
for categorical features
def plot_categorical_target_likelihood(df, column, target_column_name):
    # Calculate the mean of the target variable for each category
    likelihood =
df.groupby(column)[target_column_name].mean().sort_values(ascending=False)

    # Plot the likelihood
    plt.figure(figsize=(10, 6))
    sns.barplot(x=likelihood.index, y=likelihood.values)
    plt.title(f'Fraud Likelihood by {column}')
    plt.xlabel(column)
    plt.ylabel('Fraud Likelihood')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()

categorical_cols_train = X_train.select_dtypes(include='object').columns
```

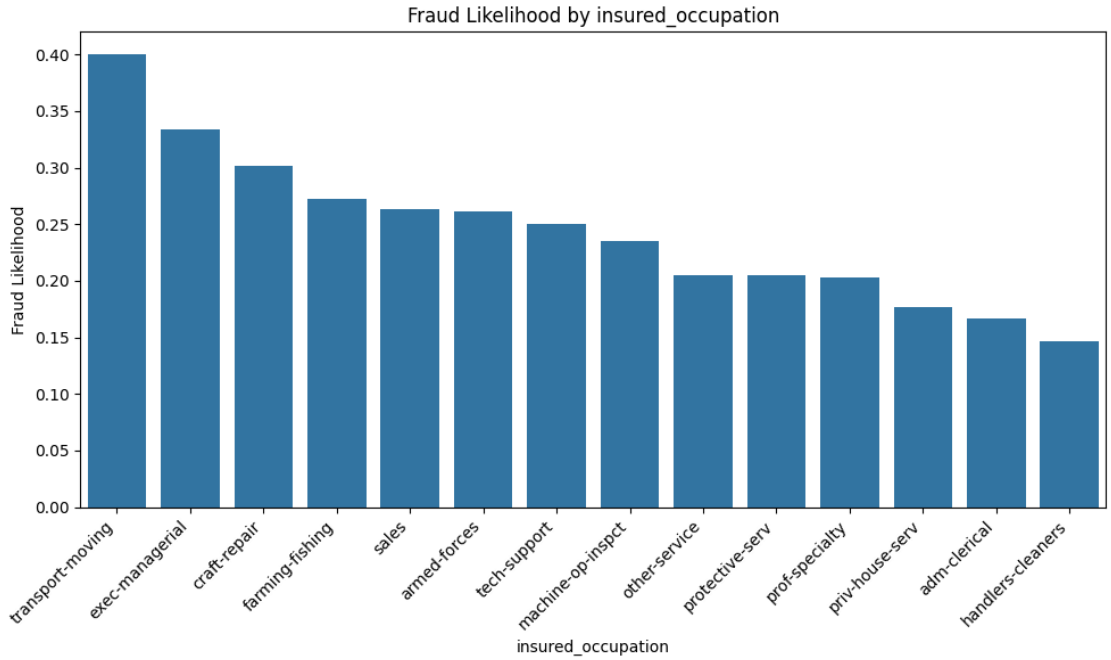
```python
# Create a combined DataFrame for plotting
train_data_for_eda = pd.concat([X_train, y_train], axis=1)

for col in categorical_cols_train:
    plot_categorical_target_likelihood(train_data_for_eda, col,
'fraud_reported')
```

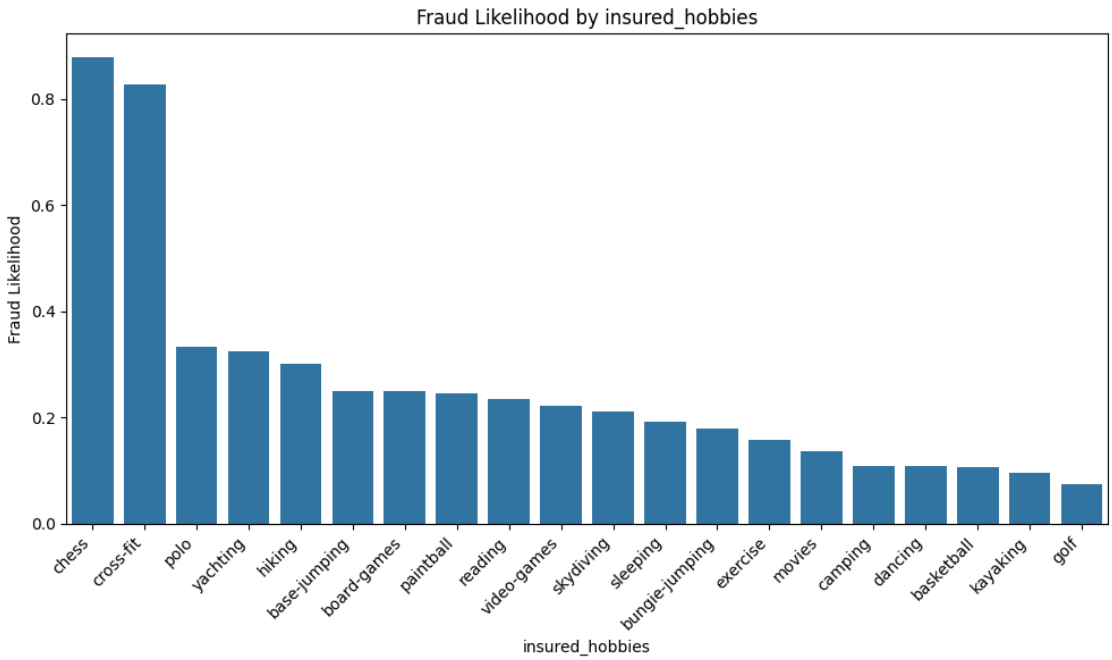


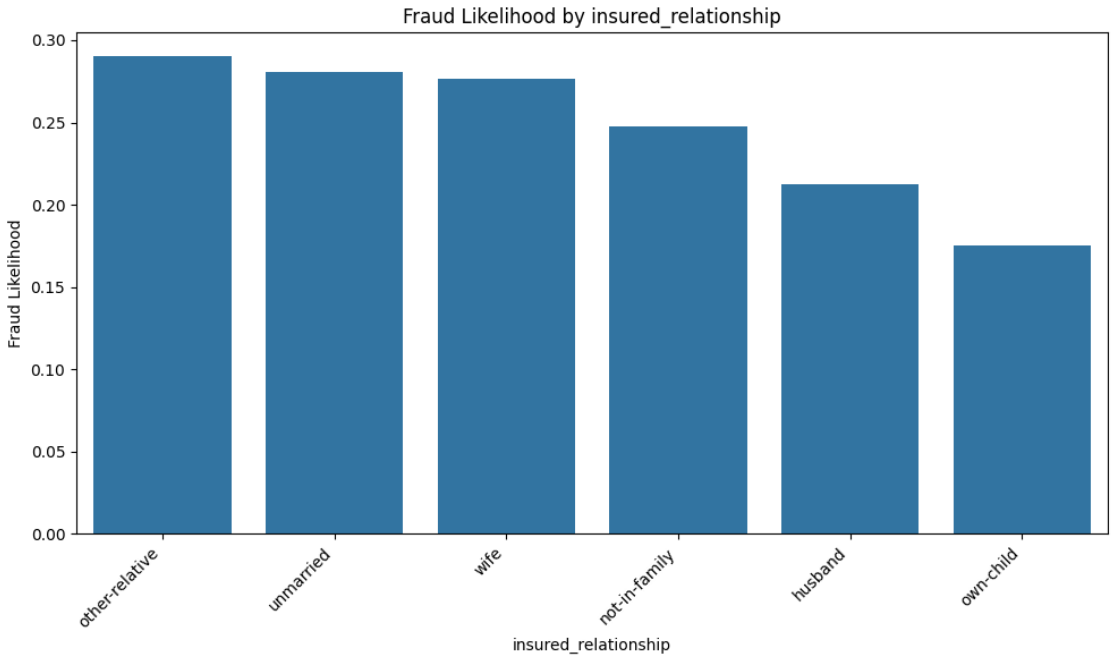


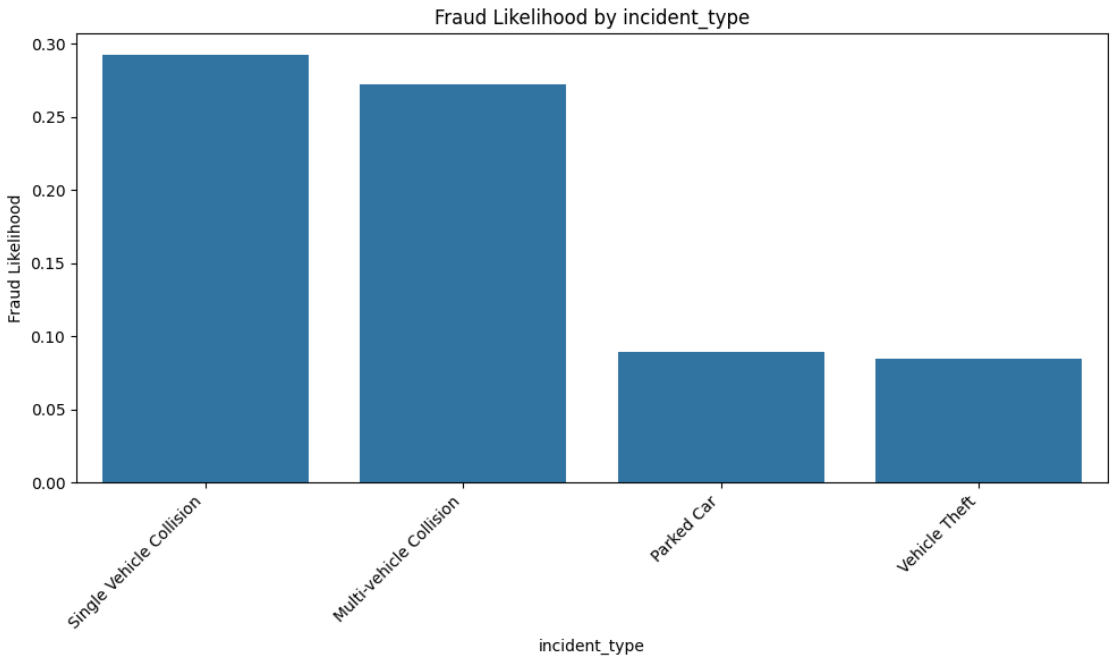

Fraud Likelihood by insured_sex
Fraud Likelihood
0.25
0.20
0.15
0.10
0.05
0.00
FEMALE
MALE
insured_sex

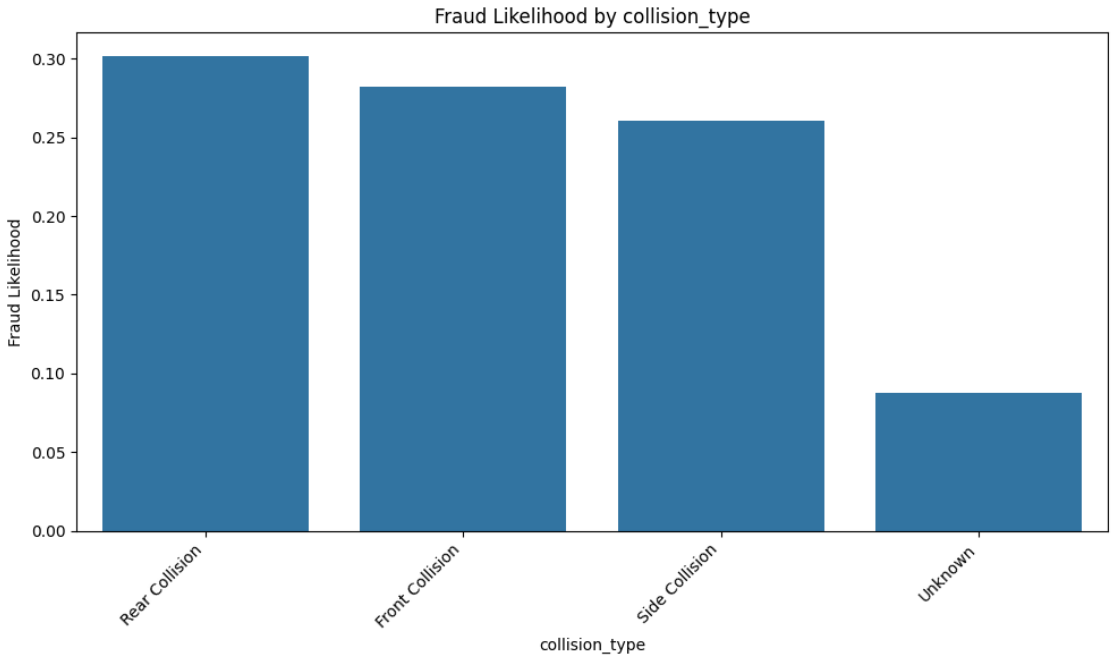

Fraud Likelihood by insured_education_level
Fraud Likelihood
0.25
0.20
0.15
0.10
0.05
0.00
MD
PhD
JD
College
Associate
Masters
High School
insured_education_level

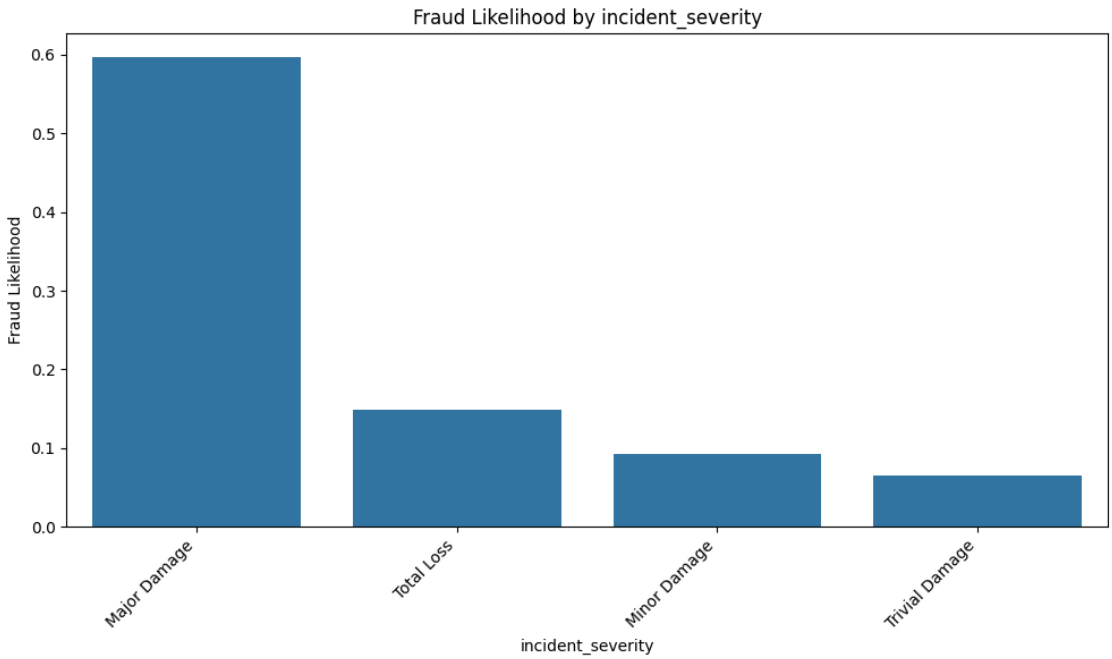

Fraud Likelihood by insured_occupation
Fraud Likelihood
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
transport-moving
exec-managerial
craft-repair
farming-fishing
sales
armed-forces
tech-support
machine-op-inspct
other-service
protective-serv
prof-specialty
priv-house-serv
adm-clerical
handlers-cleaners
insured_occupation

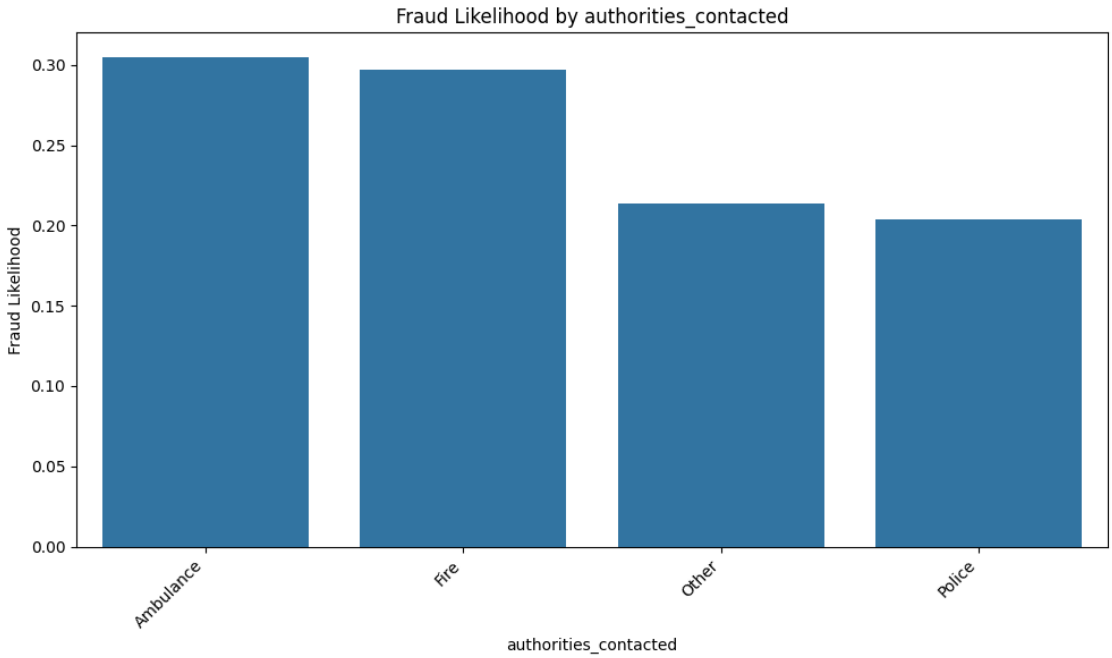

Fraud Likelihood by insured_hobbies
Fraud Likelihood
0.8
0.6
0.4
0.2
0.0
chess
cross-fit
polo
yachting
hiking
base-jumping
board-games
paintball
reading
video-games
skydiving
sleeping
bungie-jumping
exercise
movies
camping
dancing
basketball
kayaking
golf
insured_hobbies

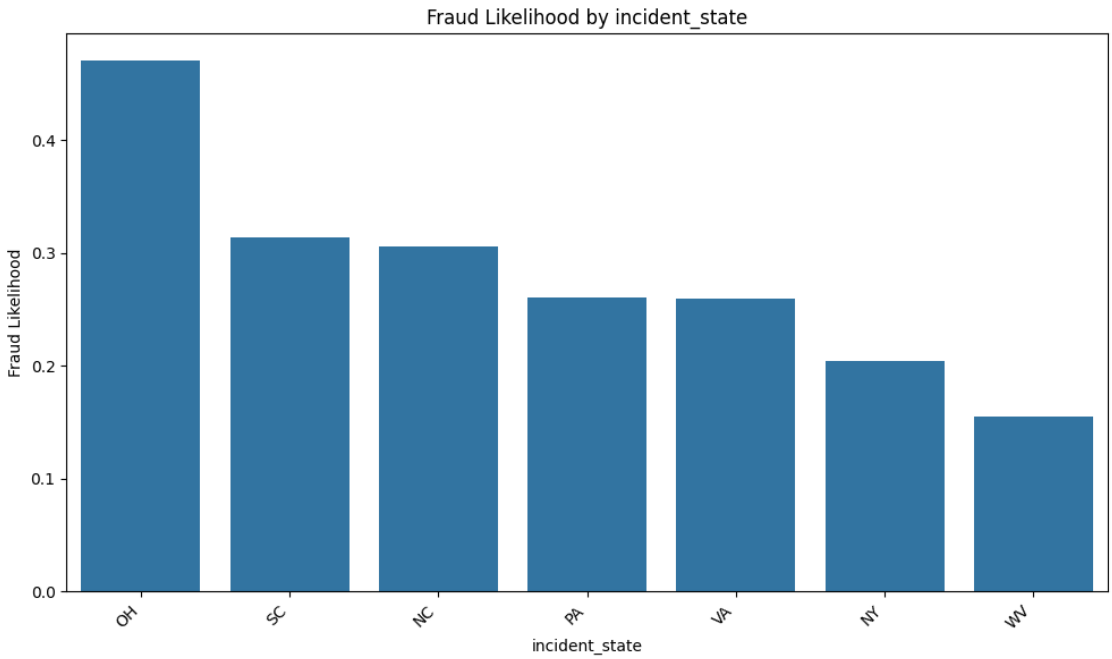

Fraud Likelihood by insured_relationship
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
other-relative
unmarried
wife
not-in-family
husband
own-child
insured_relationship

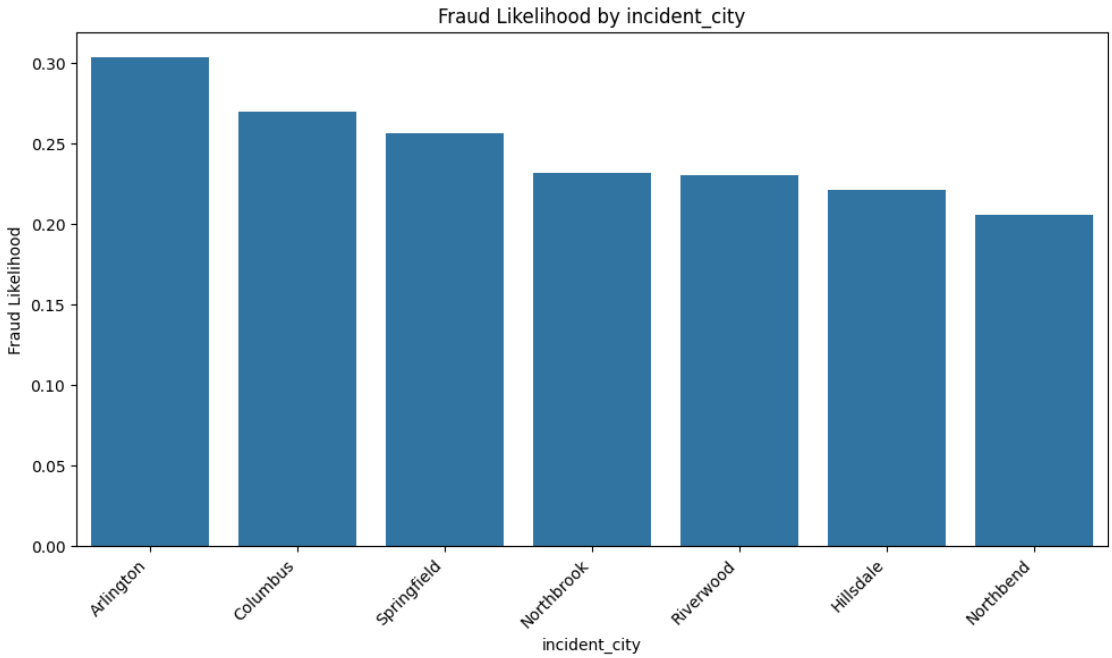

Fraud Likelihood by incident_type
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Single Vehicle Collision
Multi-vehicle Collision
Parked Car
Vehicle Theft
incident_type

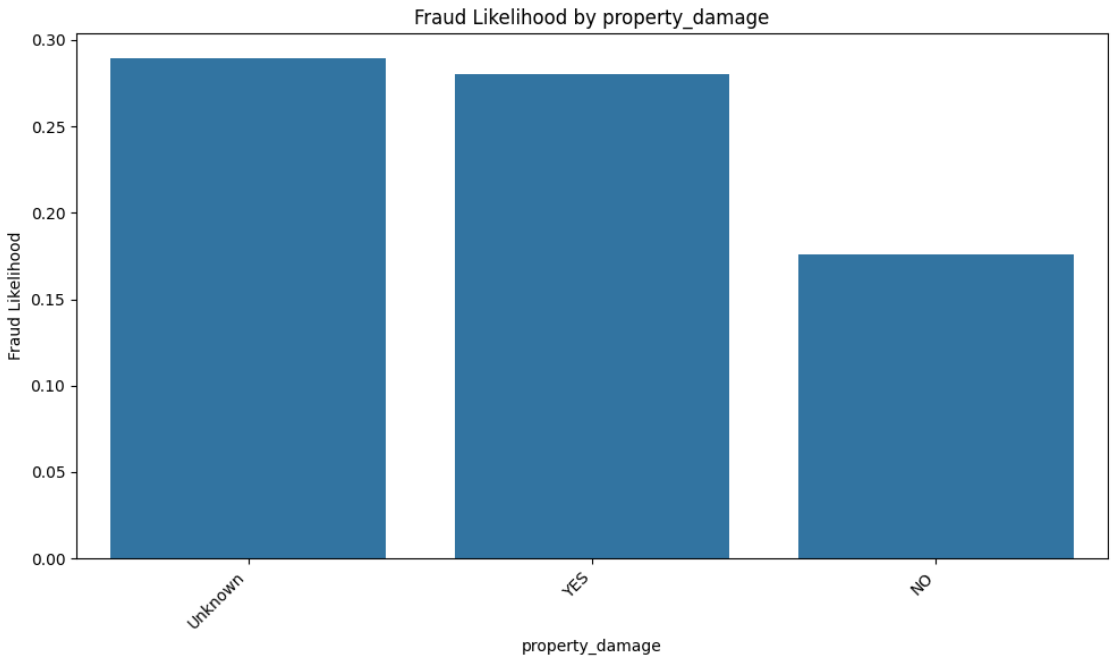

Fraud Likelihood by collision_type
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Rear Collision
Front Collision
Side Collision
Unknown
collision_type


Fraud Likelihood by incident_severity
Fraud Likelihood
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Major Damage
Total Loss
Minor Damage
Trivial Damage
incident_severity

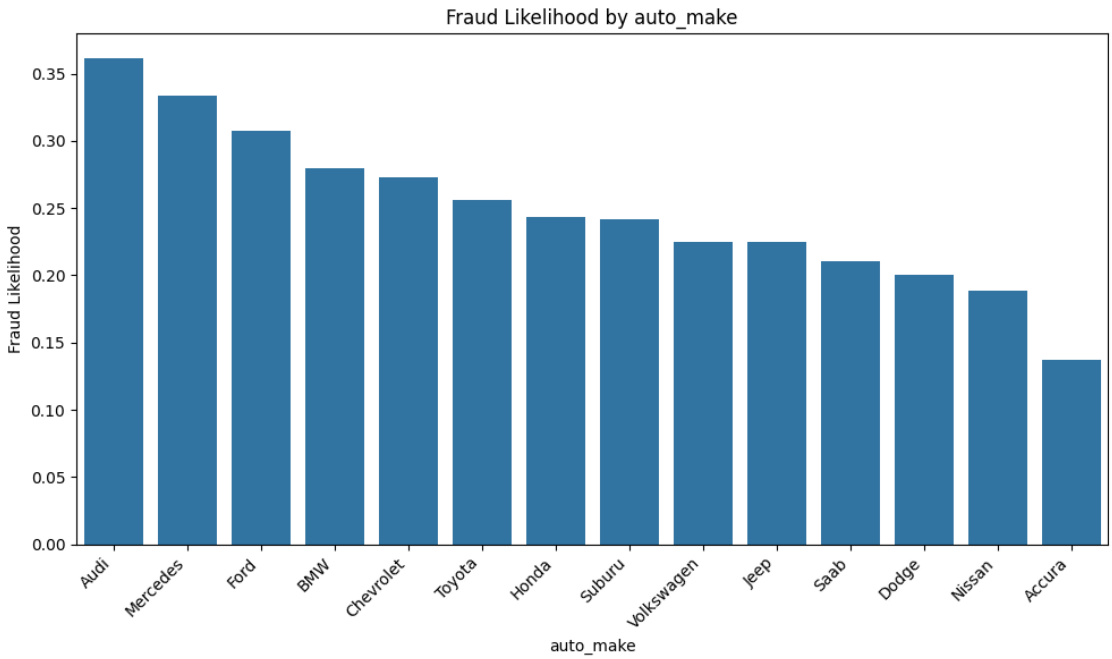

Fraud Likelihood by authorities_contacted
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Ambulance
Fire
Other
Police
authorities_contacted

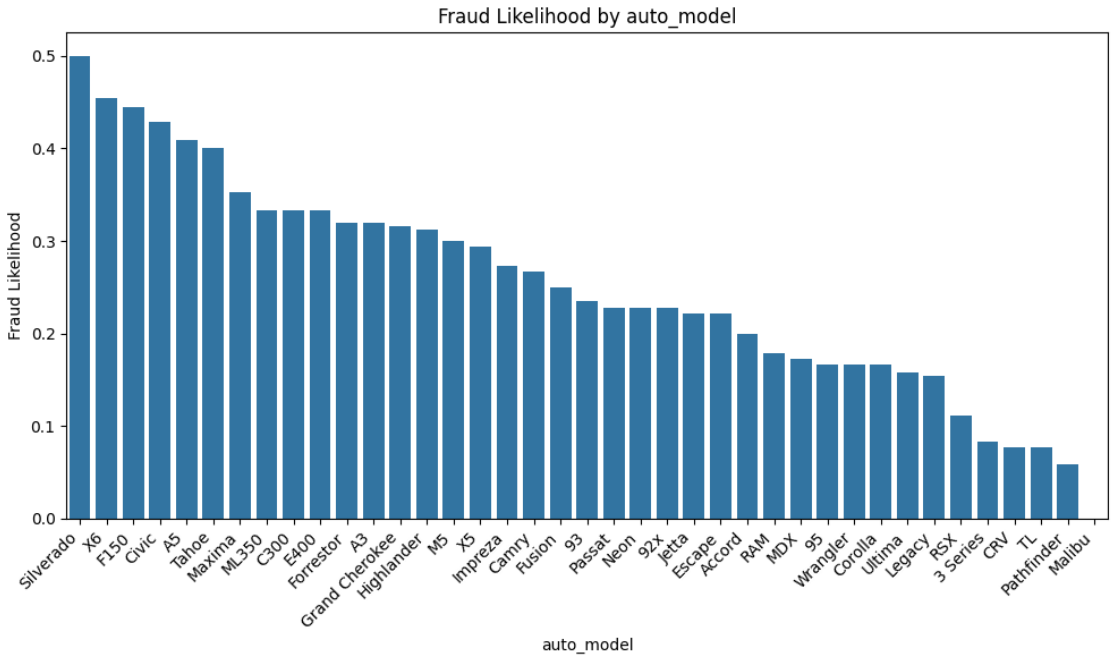

Fraud Likelihood by incident_state
Fraud Likelihood
0.4
0.3
0.2
0.1
0.0
OH
SC
NC
PA
VA
NY
WV
incident_state


Fraud Likelihood by incident_city
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Arlington
Columbus
Springfield
Northbrook
Riverwood
Hillsdale
Northbend
incident_city


Fraud Likelihood by property_damage
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Unknown
YES
NO
property_damage

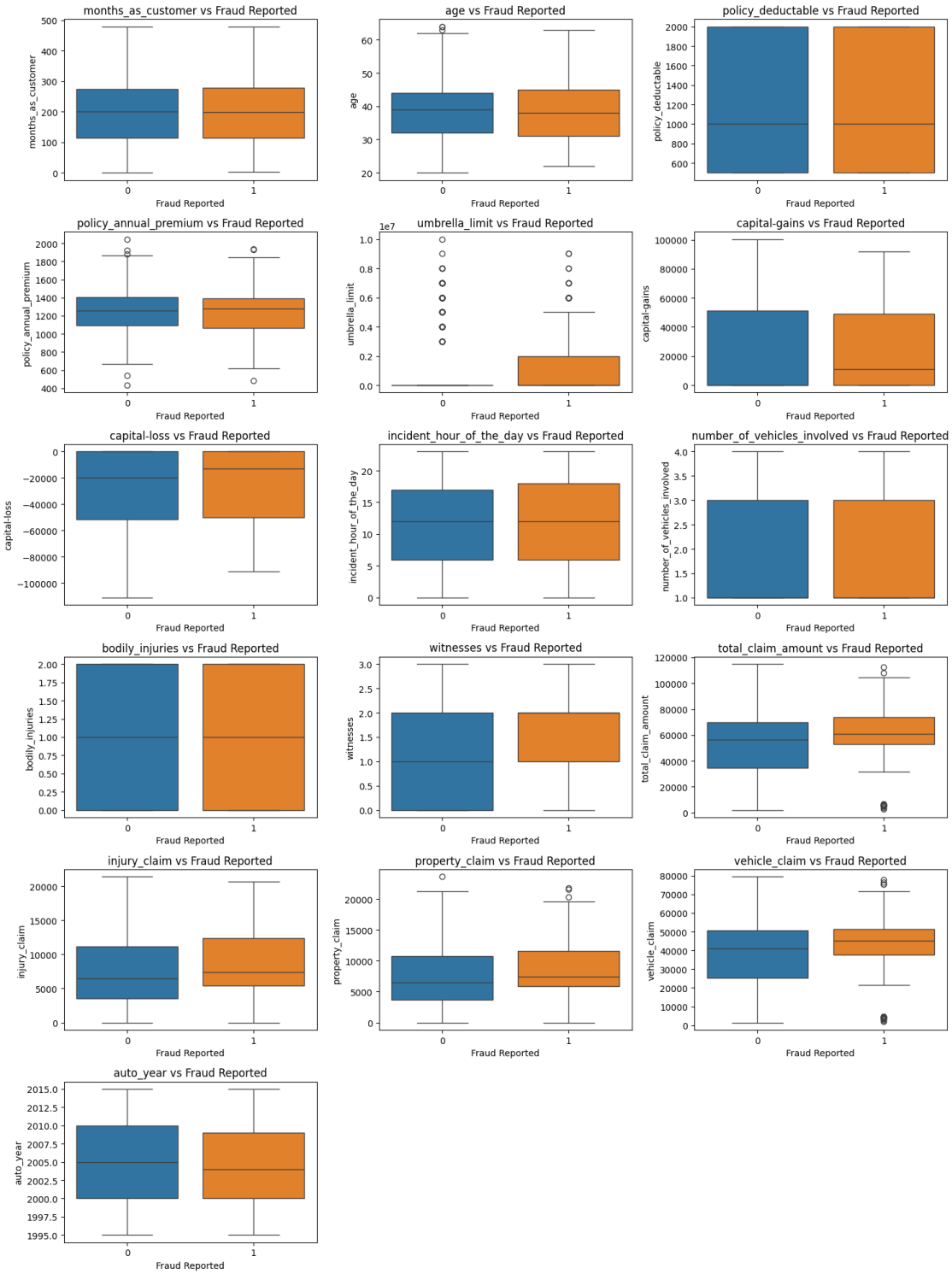
Fraud Likelihood by police_report_available
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
Unknown
NO
YES
police_report_available


Fraud Likelihood by auto_make
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Audi
Mercedes
Ford
BMW
Chevrolet
Toyota
Honda
Suburu
Volkswagen
Jeep
Saab
Dodge
Nissan
Accura
auto_make



4.4.2 *Explore the relationships between numerical features and the target variable to understand their impact on the target outcome using appropriate visualisation techniques to identify trends and potential interactions. [5 Marks]*

```
# Visualise the relationship between numerical features and the target
variable to understand their impact on the target outcome

plt.figure(figsize=(15, 20))
custom_palette = {'0': 'tab:blue', '1': 'tab:orange'}
for i, col in enumerate(numerical_cols_train):
    plt.subplot(6, 3, i + 1) # Adjust subplot grid based on number of
numerical columns
    sns.boxplot(y=train_data_for_eda[col],
x=train_data_for_eda['fraud_reported'], palette=custom_palette)
    plt.title(f'{col} vs Fraud Reported')
    plt.xlabel('Fraud Reported')
    plt.ylabel(col)
plt.tight_layout()
plt.show()
```


months_as_customer vs Fraud Reported
months_as_customer
Fraud Reported
age vs Fraud Reported
age
Fraud Reported
policy_deductable vs Fraud Reported
policy_deductable
Fraud Reported
policy_annual_premium vs Fraud Reported
policy_annual_premium
Fraud Reported
umbrella_limit vs Fraud Reported
1e7
umbrella_limit
Fraud Reported
capital-gains vs Fraud Reported
capital-gains
Fraud Reported
capital-loss vs Fraud Reported
capital-loss
Fraud Reported
incident_hour_of_the_day vs Fraud Reported
incident_hour_of_the_day
Fraud Reported
number_of_vehicles_involved vs Fraud Reported
number_of_vehicles_involved
Fraud Reported
bodily_injuries vs Fraud Reported
bodily_injuries
Fraud Reported
witnesses vs Fraud Reported
witnesses
Fraud Reported
total_claim_amount vs Fraud Reported
total_claim_amount
Fraud Reported
injury_claim vs Fraud Reported
injury_claim
Fraud Reported
property_claim vs Fraud Reported
property_claim
Fraud Reported
vehicle_claim vs Fraud Reported
vehicle_claim
Fraud Reported
auto_year vs Fraud Reported
auto_year
Fraud Reported

5. EDA on validation data [OPTIONAL]

5.1 Perform univariate analysis

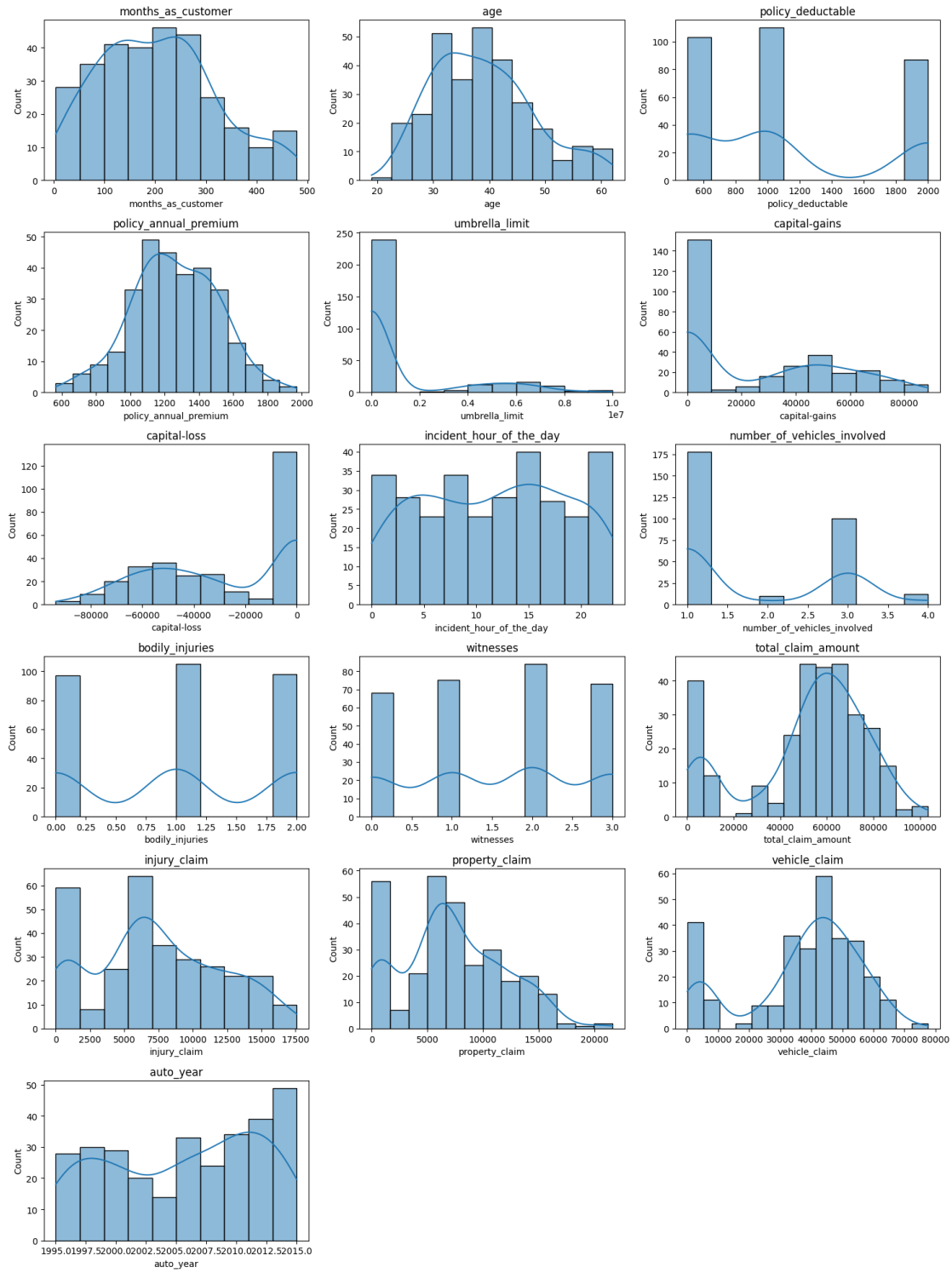
5.1.1 *Identify and select numerical columns from training data for univariate analysis.*

```
# Select numerical columns
numerical_cols_test = X_test.select_dtypes(include=np.number).columns
numerical_cols_test
```

```
Index(['months_as_customer', 'age', 'policy_deductable',
       'policy_annual_premium', 'umbrella_limit', 'capital-gains',
       'capital-loss', 'incident_hour_of_the_day',
       'number_of_vehicles_involved', 'bodily_injuries', 'witnesses',
       'total_claim_amount', 'injury_claim', 'property_claim',
'vehicle_claim',
       'auto_year'],
      dtype='object')
```

5.1.2 *Visualise the distribution of selected numerical features using appropriate plots to understand their characteristics.*

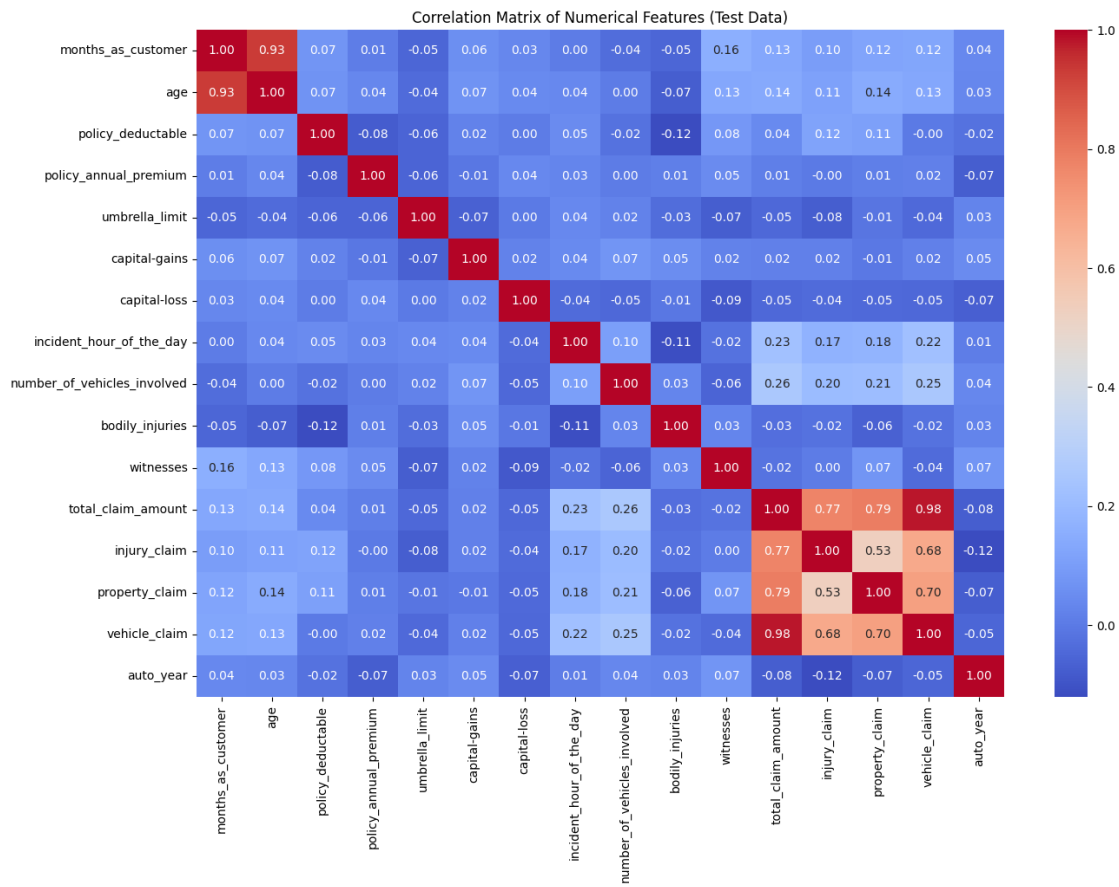
```
# Plot all the numerical columns to understand their distribution
plt.figure(figsize=(15, 20))
for i, col in enumerate(numerical_cols_test):
    plt.subplot(6, 3, i + 1) # Adjust subplot grid based on number of
numerical columns
    sns.histplot(X_test[col], kde=True)
    plt.title(col)
plt.tight_layout()
plt.show()
```



5.2 Perform correlation analysis

Investigate the relationships between numerical features to identify potential multicollinearity or dependencies. Visualise the correlation structure using an appropriate method to gain insights into feature relationships.

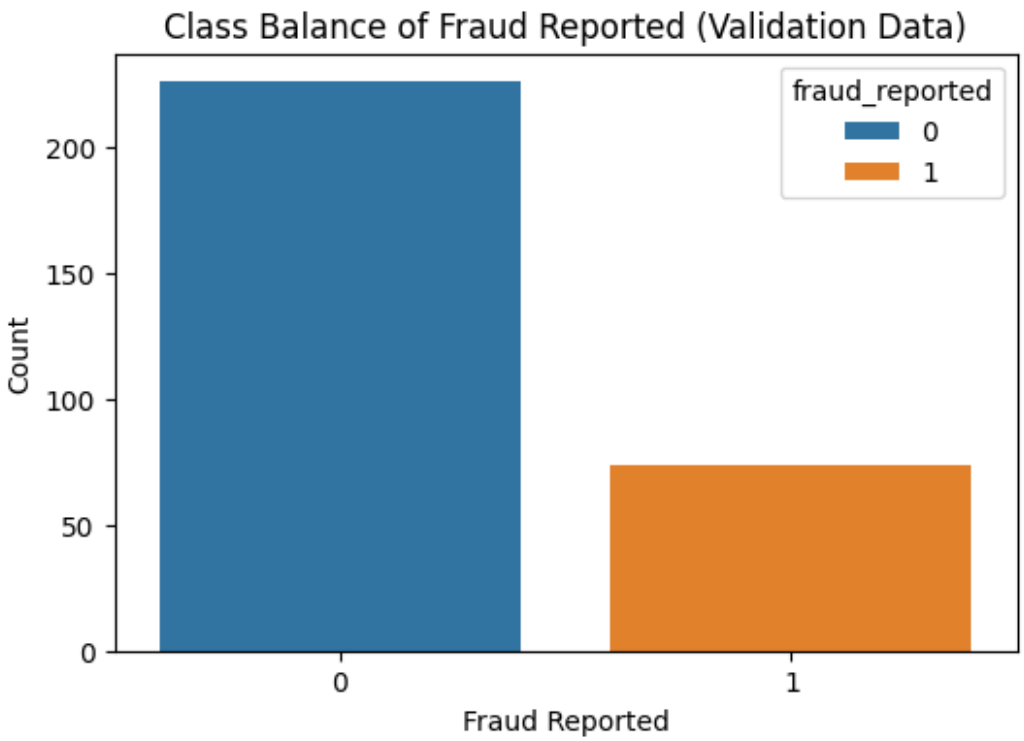
```python
# Create correlation matrix for numerical columns
corr_matrix_test = X_test[numerical_cols_test].corr()

# Plot Heatmap of the correlation matrix
plt.figure(figsize=(15, 10))
sns.heatmap(corr_matrix_test, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Matrix of Numerical Features (Test Data)')
plt.show()
```



5.3 Check class balance

Examine the distribution of the target variable to identify potential class imbalances. Visualise the distribution for better understanding.

```python
# Plot a bar chart to check class balance
plt.figure(figsize=(6, 4))
sns.countplot(x=y_test, hue=y_test)
plt.title('Class Balance of Fraud Reported (Validation Data)')
plt.xlabel('Fraud Reported')
plt.ylabel('Count')
plt.show()
```

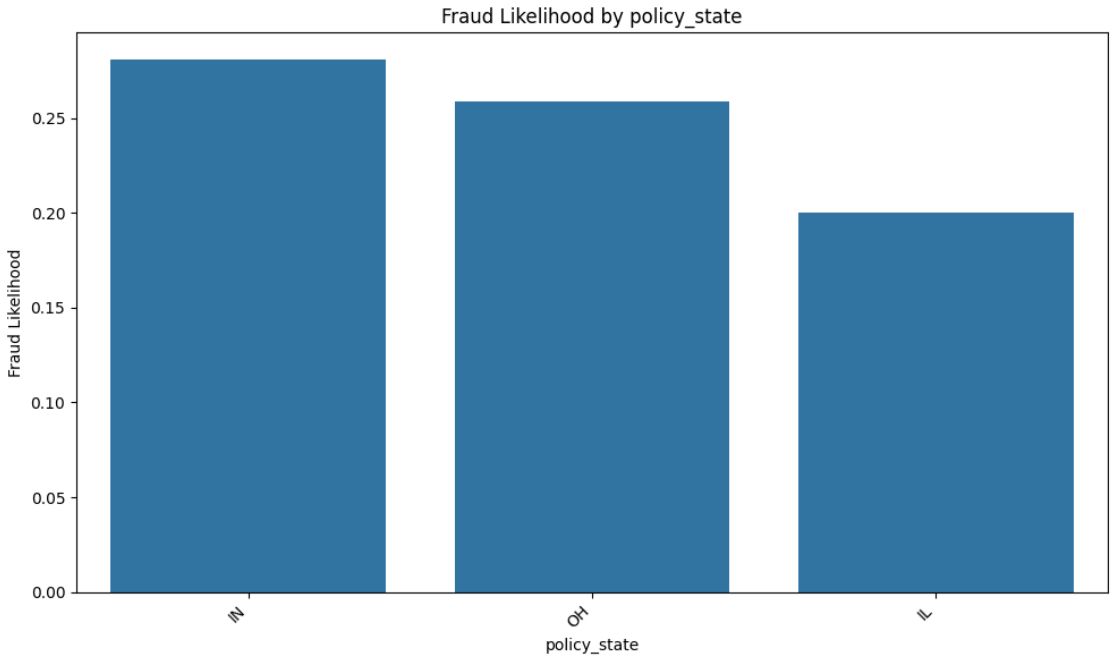


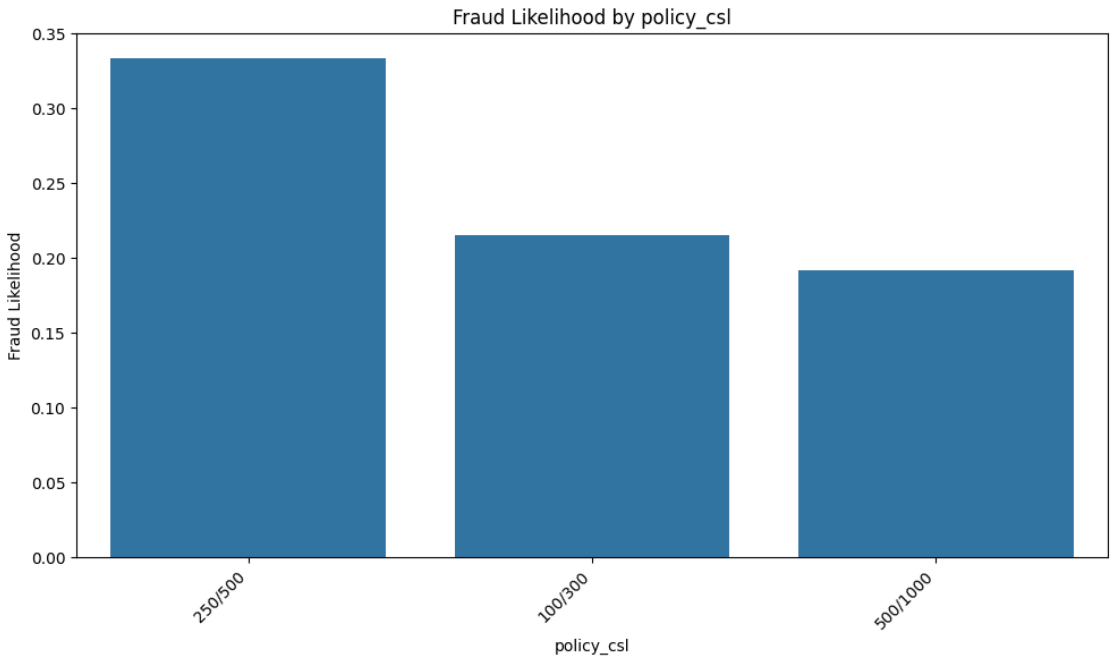
5.4 Perform bivariate analysis

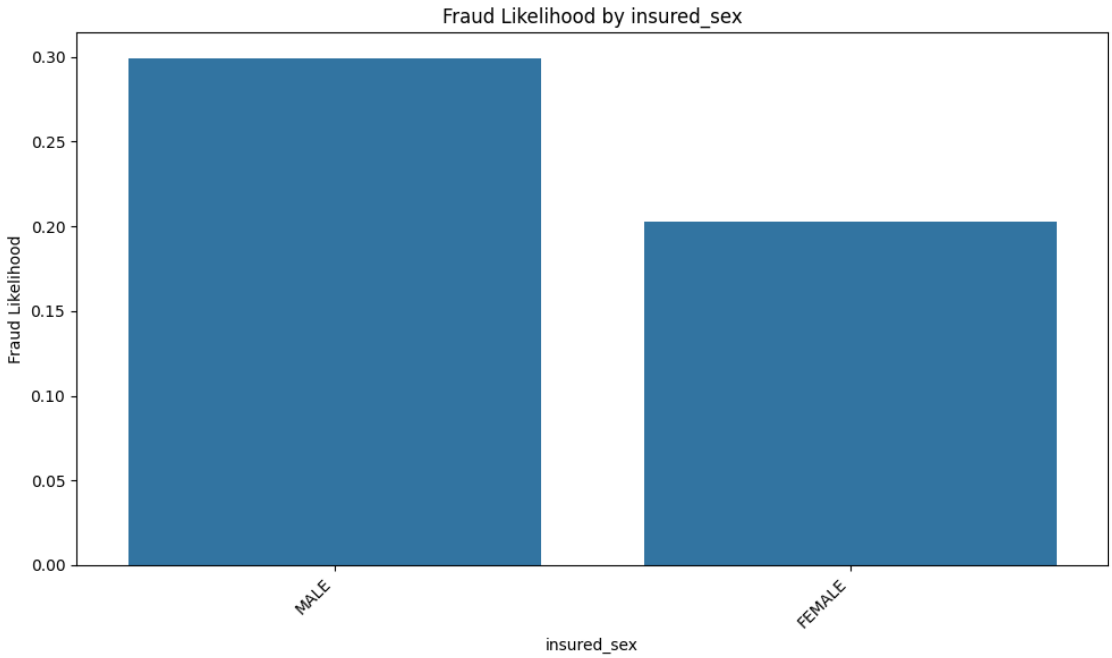
5.4.1 Target likelihood analysis for categorical variables.

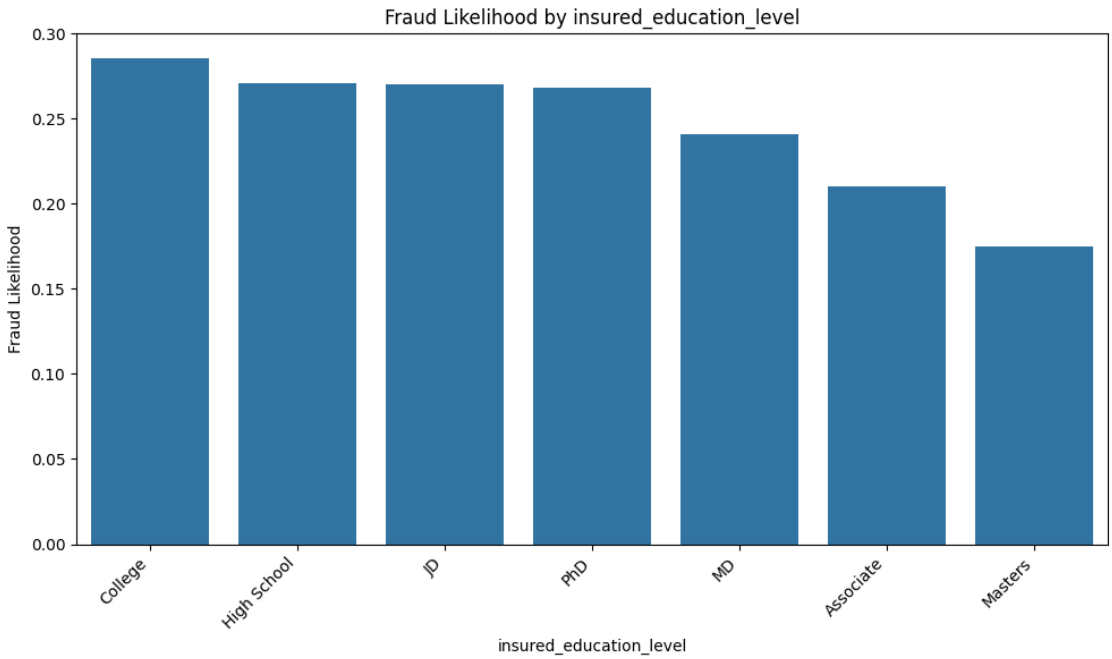
Investigate the relationships between categorical features and the target variable by analysing the target event likelihood (for the `'Y'` event) for each level of every relevant categorical feature. Through this analysis, identify categorical features that do not contribute much in explaining the variation in the target variable.

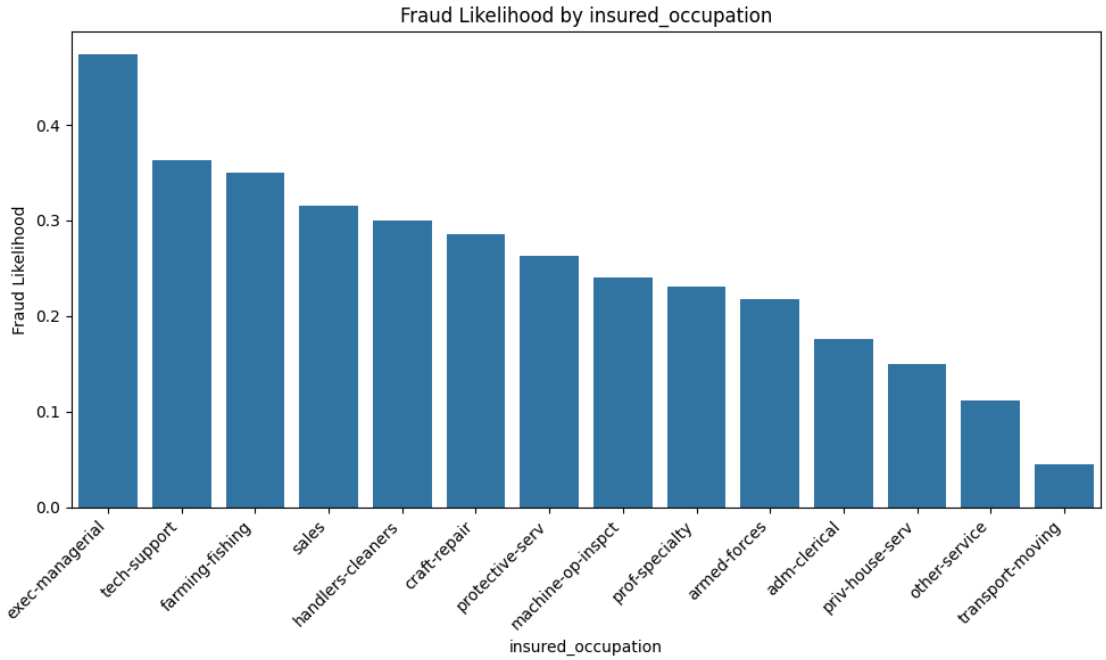
```
# Write a function to calculate and analyse the target variable likelihood
for categorical features
categorical_cols_test = X_test.select_dtypes(include='object').columns

# Create a combined DataFrame for plotting
test_data_for_eda = pd.concat([X_test, y_test], axis=1)

for col in categorical_cols_test:
    plot_categorical_target_likelihood(test_data_for_eda, col,
'fraud_reported')
```

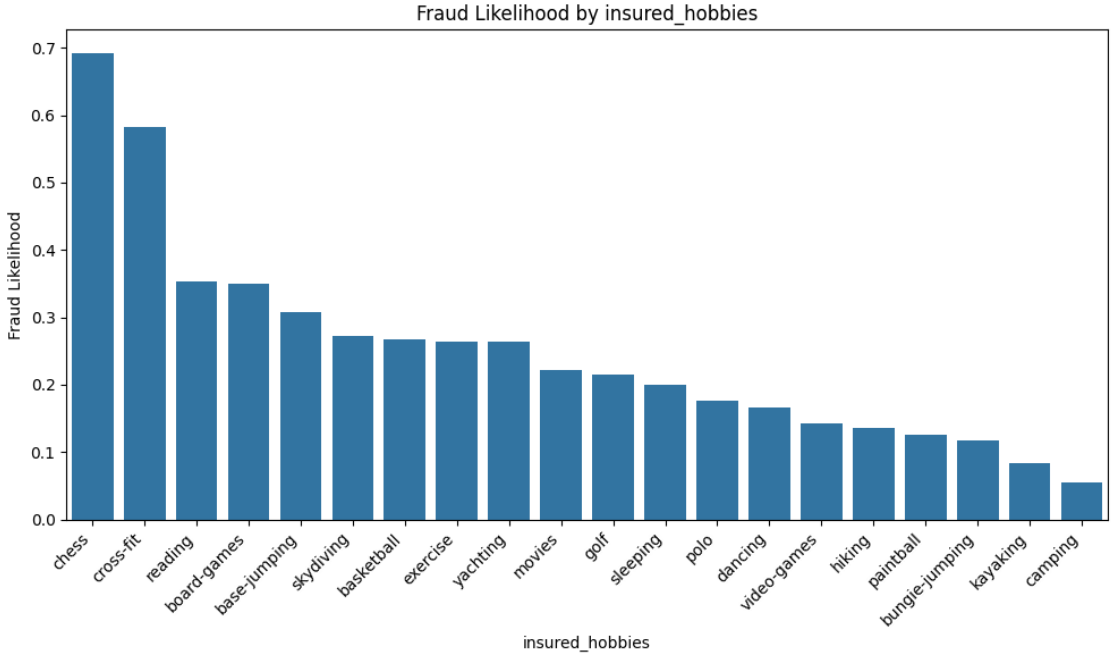

Fraud Likelihood by policy_state
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
IN
OH
IL
policy_state

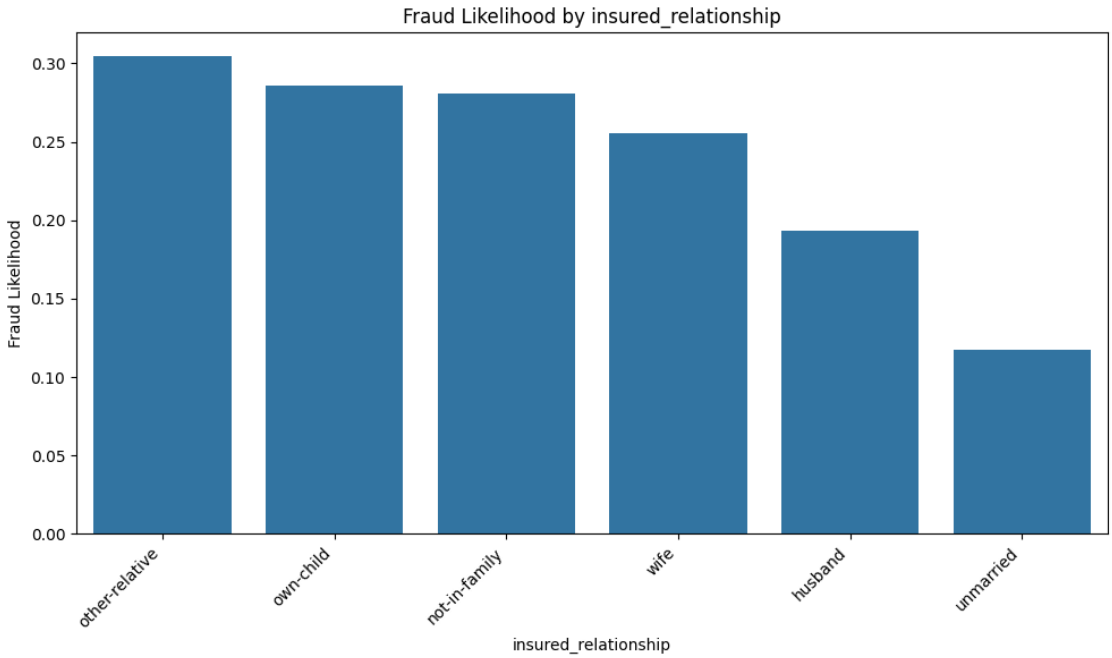

Fraud Likelihood by policy_csl
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
250/500
100/300
500/1000
policy_csl

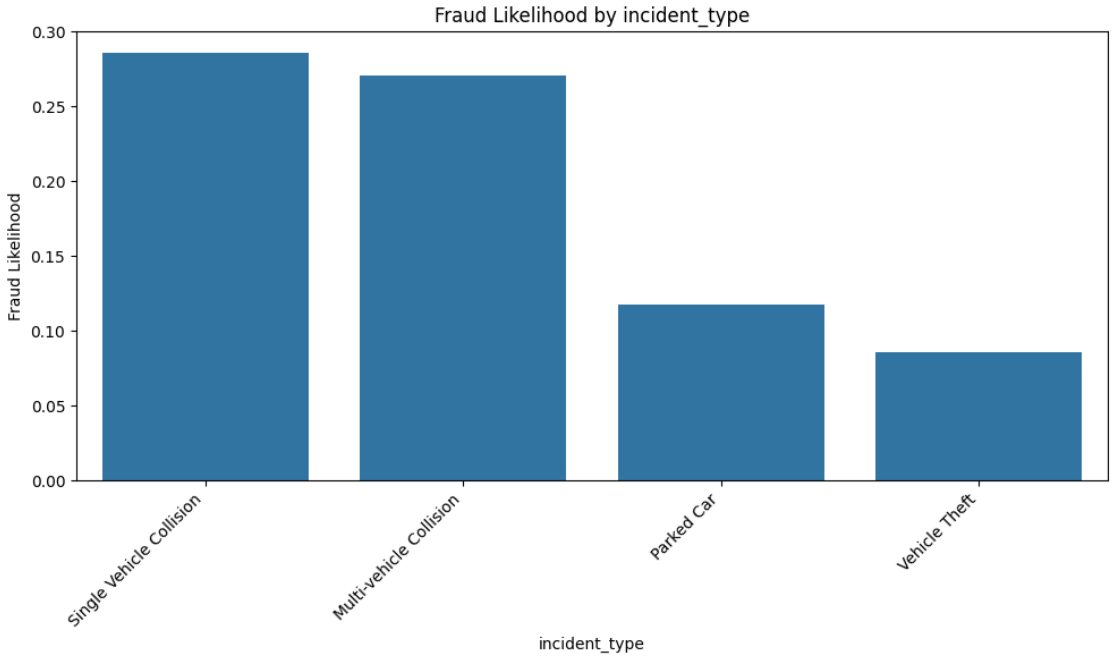

Fraud Likelihood by insured_sex
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
MALE
FEMALE
insured_sex

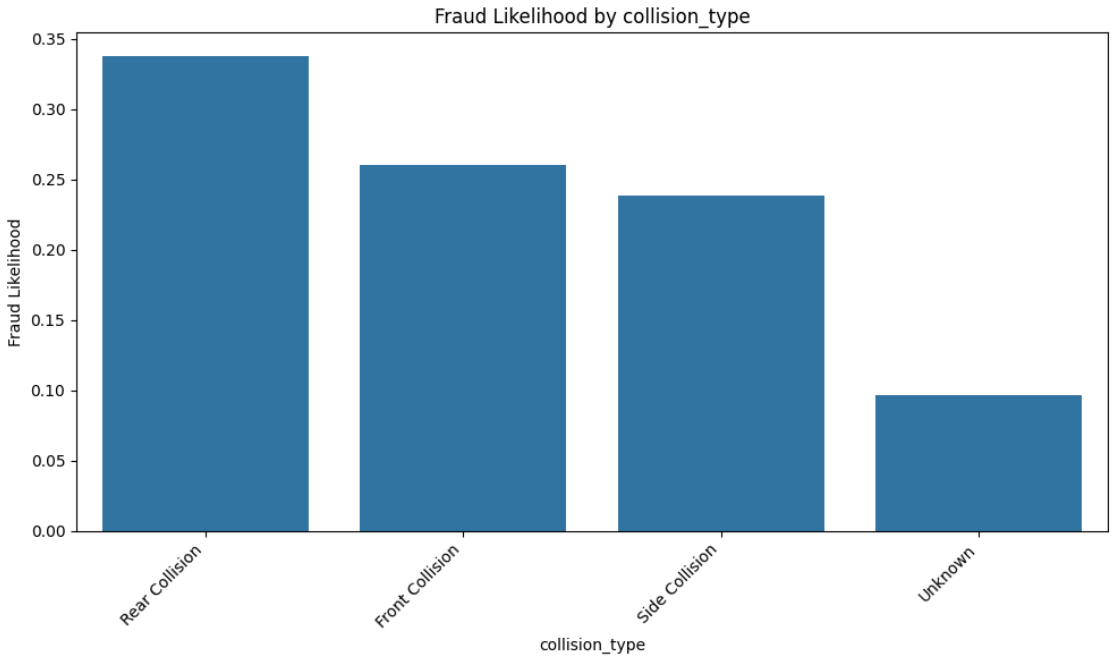

Fraud Likelihood by insured_education_level
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
College
High School
JD
PhD
MD
Associate
Masters
insured_education_level

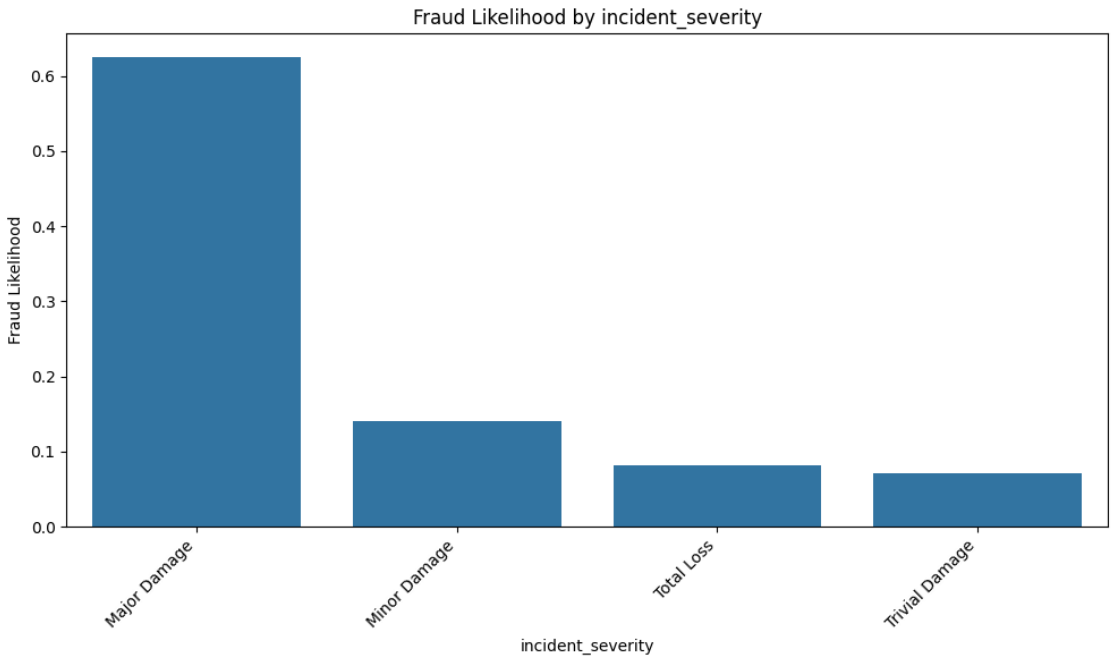

Fraud Likelihood by insured_occupation
Fraud Likelihood
0.0
0.1
0.2
0.3
0.4
exec-managerial
tech-support
farming-fishing
sales
handlers-cleaners
craft-repair
protective-serv
machine-op-inspct
prof-specialty
armed-forces
adm-clerical
priv-house-serv
other-service
transport-moving
insured_occupation

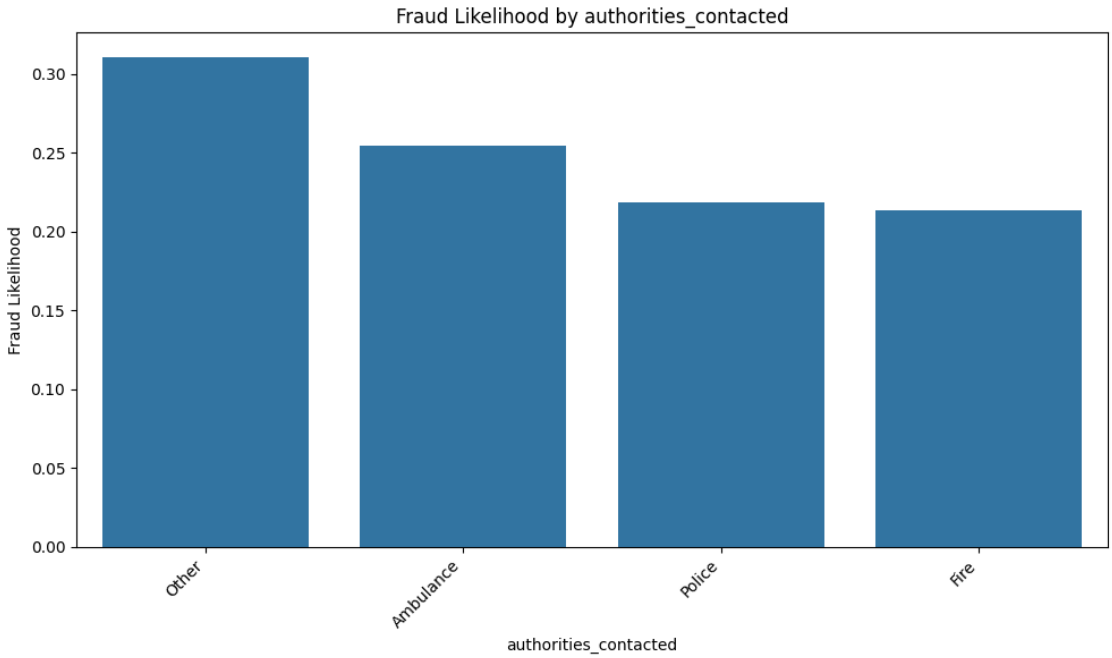

Fraud Likelihood by insured_hobbies
Fraud Likelihood
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
chess
cross-fit
reading
board-games
base-jumping
skydiving
basketball
exercise
yachting
movies
golf
sleeping
polo
dancing
video-games
hiking
paintball
bungie-jumping
kayaking
camping
insured_hobbies

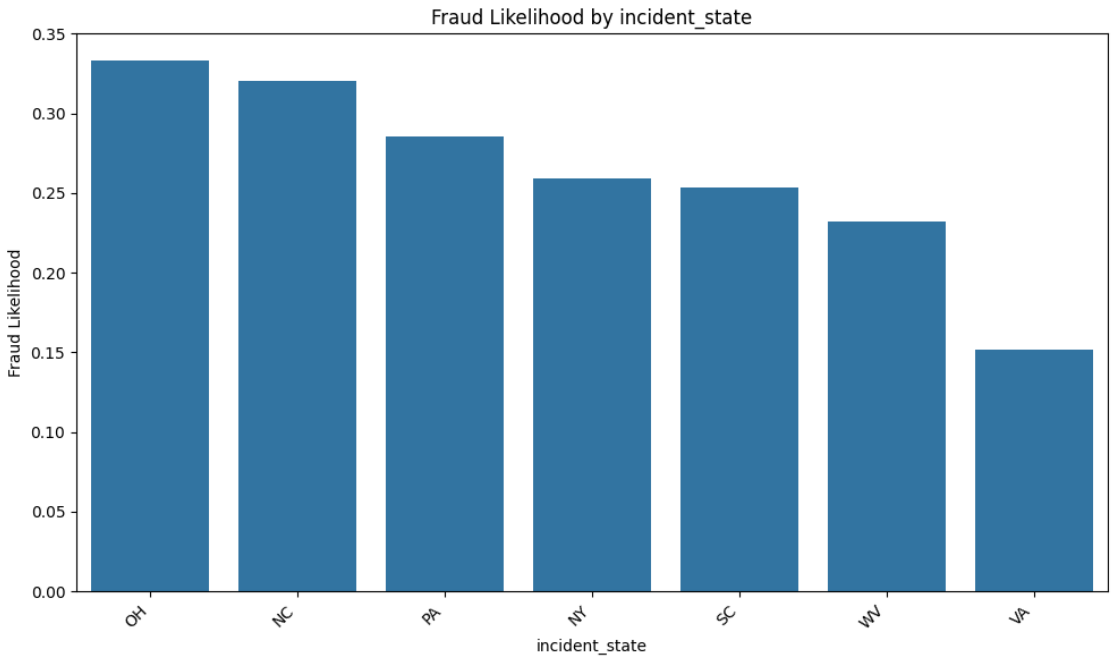

Fraud Likelihood by insured_relationship
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
other-relative
own-child
not-in-family
wife
husband
unmarried
insured_relationship

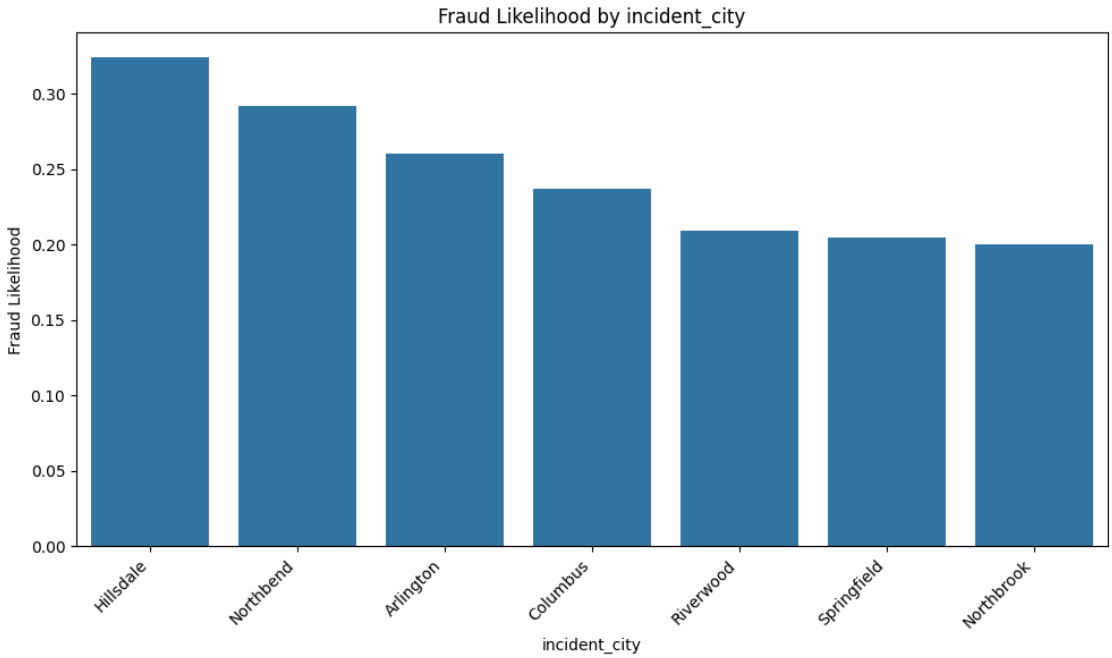

Fraud Likelihood by incident_type
Fraud Likelihood
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Single Vehicle Collision
Multi-vehicle Collision
Parked Car
Vehicle Theft
incident_type

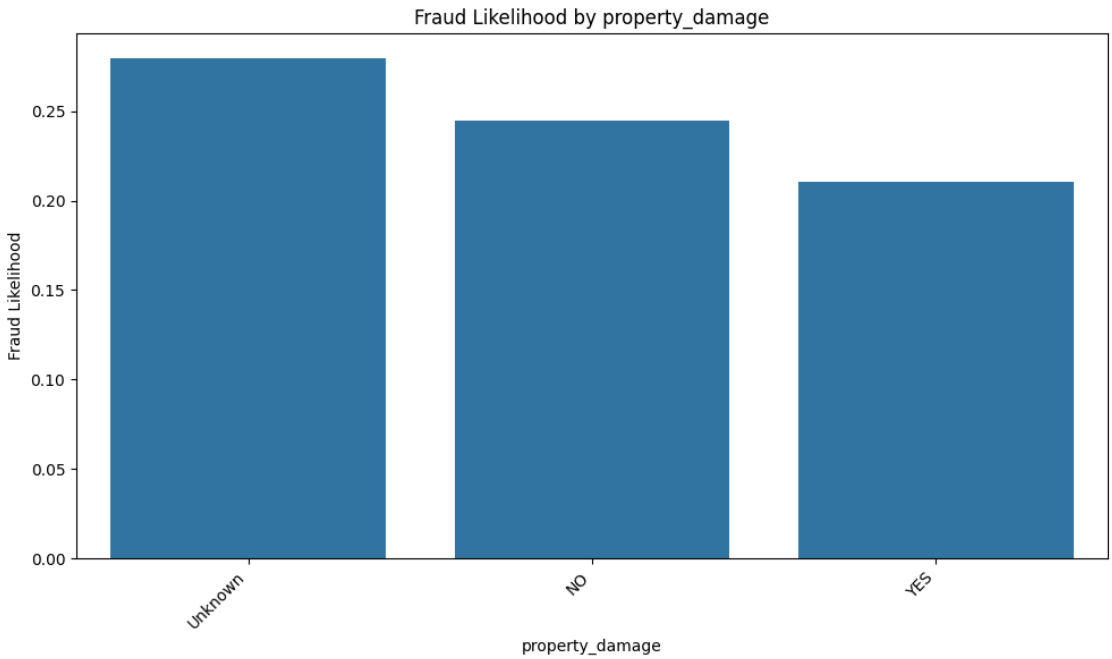

Fraud Likelihood by collision_type
Fraud Likelihood
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
Rear Collision
Front Collision
Side Collision
Unknown
collision_type


Fraud Likelihood by incident_severity
Fraud Likelihood
0.6
0.5
0.4
0.3
0.2
0.1
0.0
Major Damage
Minor Damage
Total Loss
Trivial Damage
incident_severity

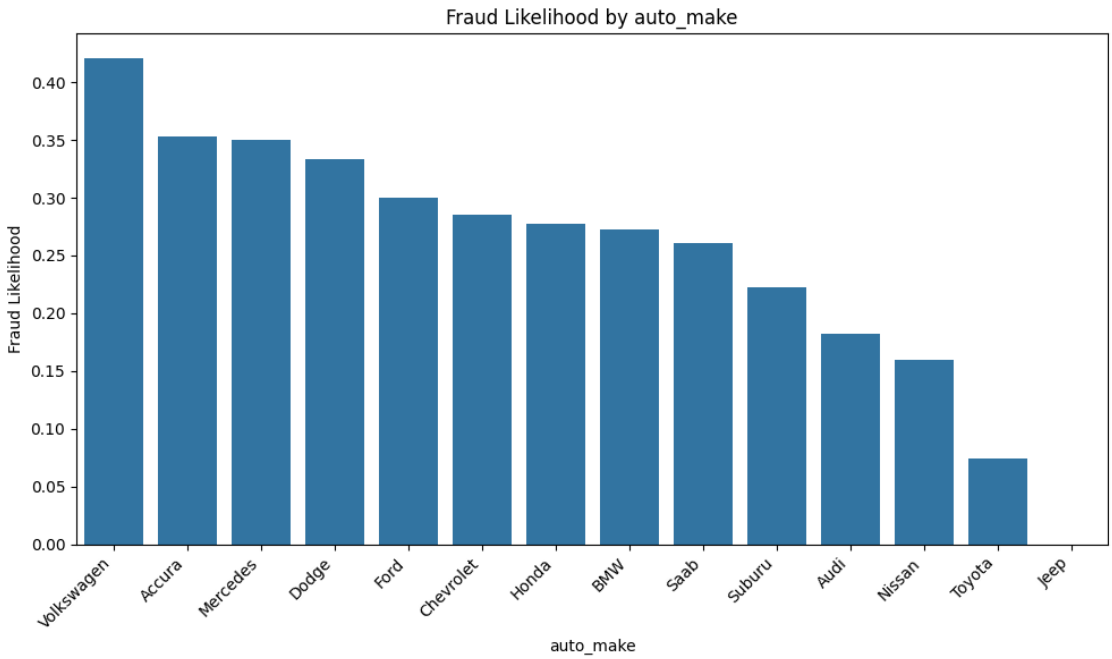

Fraud Likelihood by authorities_contacted
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Other
Ambulance
Police
Fire
authorities_contacted

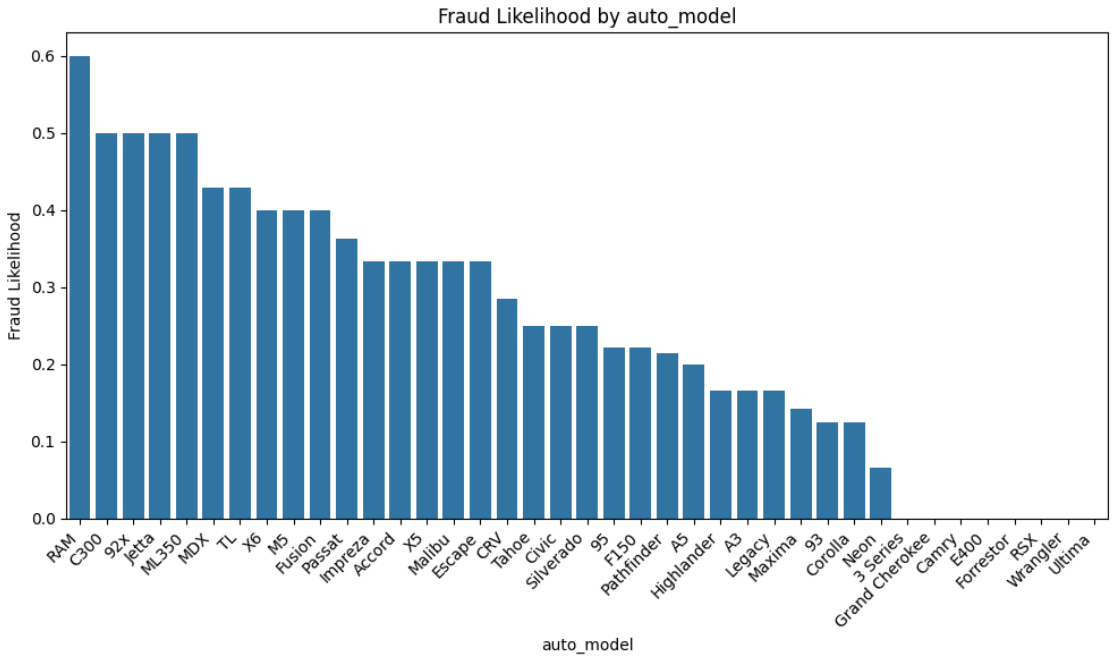

Fraud Likelihood by incident_state
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
OH
NC
PA
NY
SC
WV
VA
incident_state


Fraud Likelihood by incident_city
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
0.30
Hillsdale
Northbend
Arlington
Columbus
Riverwood
Springfield
Northbrook
incident_city


Fraud Likelihood by property_damage
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
Unknown
NO
YES
property_damage

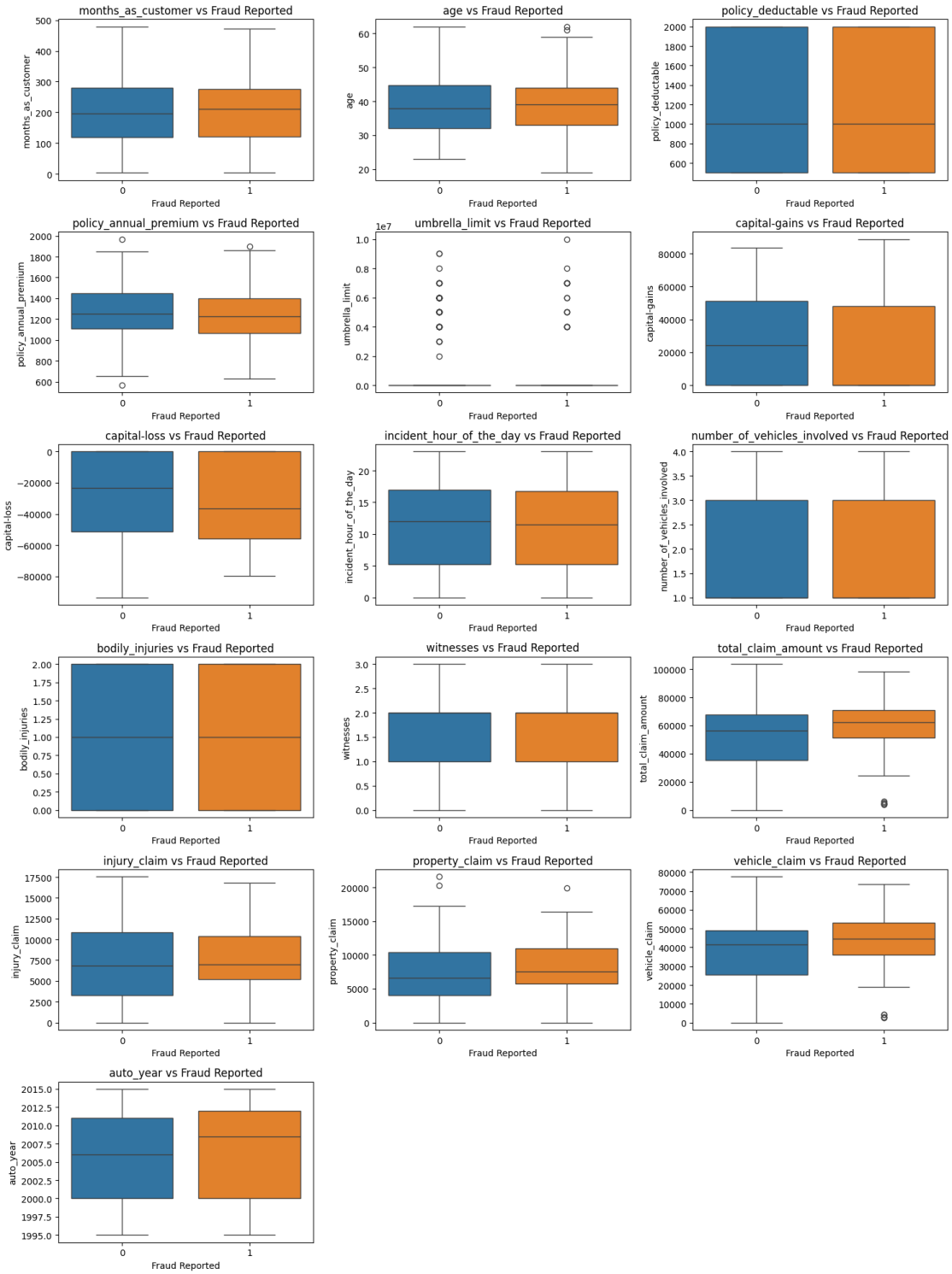
Fraud Likelihood by police_report_available
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
Unknown
NO
YES
police_report_available


Fraud Likelihood by auto_make
Fraud Likelihood
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
0.40
Volkswagen
Accura
Mercedes
Dodge
Ford
Chevrolet
Honda
BMW
Saab
Suburu
Audi
Nissan
Toyota
Jeep
auto_make



5.4.2 *Explore the relationships between numerical features and the target variable to understand their impact on the target outcome. Utilise appropriate visualisation techniques to identify trends and potential interactions.*

```
# Visualise the relationship between numerical features and the target
variable to understand their impact on the target outcome

plt.figure(figsize=(15, 20))
custom_palette = {'0': 'tab:blue', '1': 'tab:orange'}
for i, col in enumerate(numerical_cols_test):
    plt.subplot(6, 3, i + 1)  # Adjust subplot grid based on number of
numerical columns
    sns.boxplot(y=test_data_for_eda[col],
x=test_data_for_eda['fraud_reported'], palette=custom_palette)
    plt.title(f'{col} vs Fraud Reported')
    plt.xlabel('Fraud Reported')
    plt.ylabel(col)
plt.tight_layout()
plt.show()
```



6. Feature Engineering [25 marks]

6.1 Perform resampling [3 Marks]

Handle class imbalance in the training data by applying resampling technique.

Use the **RandomOverSampler** technique to balance the data and handle class imbalance. This method increases the number of samples in the minority class by randomly duplicating them, creating synthetic data points with similar characteristics. This helps prevent the model from being biased toward the majority class and improves its ability to predict the minority class more accurately.

Note: You can try other resampling techniques to handle class imbalance

```
# Import RandomOverSampler from imblearn library
from imblearn.over_sampling import RandomOverSampler

# Perform resampling on training data
ros = RandomOverSampler(random_state=31)
X_train_resampled, y_train_resampled = ros.fit_resample(X_train, y_train)

print("Class distribution before resampling:")
print(y_train.value_counts())

print("\nClass distribution after resampling:")
print(y_train_resampled.value_counts())
```

```
Class distribution before resampling:
fraud_reported
0    527
1    173
Name: count, dtype: int64

Class distribution after resampling:
fraud_reported
0    527
1    527
Name: count, dtype: int64
```

6.2 Feature Creation [4 marks]

Create new features from existing ones to enhance the model's ability to capture patterns in the data. This may involve deriving features from date/time columns, combining features, or creating interaction terms.

```
#Checking the value counts of features that can be derived from incident_date
df['incident_date'].dt.year.value_counts()
```

```
incident_date
2015    1000
Name: count, dtype: int64
```

```
#Checking the value counts of features that can be derived from incident_date
df['incident_date'].dt.month.value_counts()
```

```
incident_date
1    516
```

```
2    472
3     12
Name: count, dtype: int64
```

```
#Since all incidents are recorded in year 2015, we can set the max_year to
2015
max_year = df['incident_date'].dt.year.max()
```

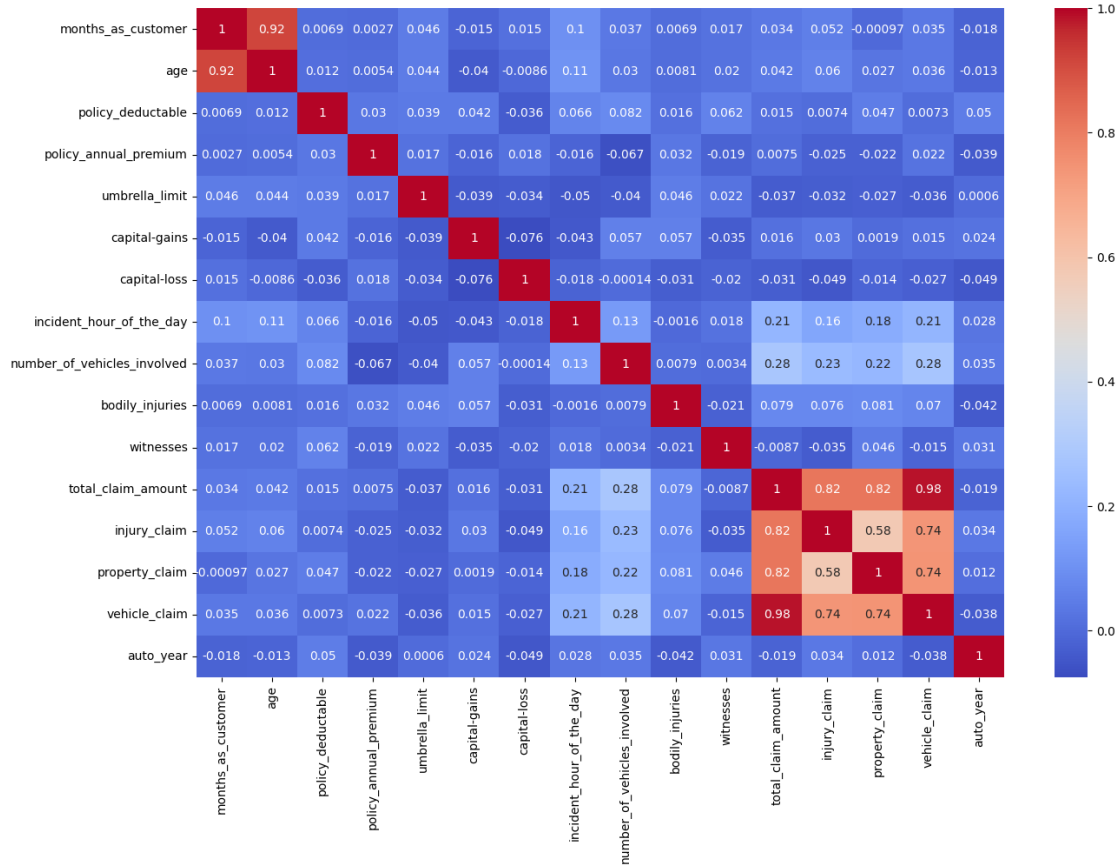
```
# Create new features based on your understanding for both training and
validation data
#As noticed above, derivable feature year has a single value and is not
particularly useful for analysis.
X_train_resampled['incident_month'] =
X_train_resampled['incident_date'].dt.month

#calculate the age of the vehicle which could be a useful predictor

X_train_resampled['auto_age'] = max_year - X_train_resampled['auto_year']

#New features for validation data

X_test['incident_month'] = X_test['incident_date'].dt.month
X_test['auto_age'] = max_year - X_test['auto_year']
```

6.3 Handle redundant columns [3 marks]

Analyse the data to identify features that may be redundant or contribute minimal information toward predicting the target variable and drop them.

- You can consider features that exhibit high correlation with other variables, which you may have observed during the EDA phase.
- Features that don't strongly influence the prediction, which you may have observed during the EDA phase.
- Categorical columns with low value counts for some levels can be remapped to reduce number of unique levels, and features with very high counts for just one level may be removed, as they resemble unique identifier columns and do not provide substantial predictive value.
- Additionally, eliminate any columns from which the necessary features have already been extracted in the preceding step.

```
# Plot Heatmap of the correlation matrix
plt.figure(figsize=(15,10))
sns.heatmap(corr_matrix_train, annot=True, cmap='coolwarm')
plt.show()
```




```
# Drop redundant columns from training and validation data
# Features that exhibit high correlation are months_as_customer vs age,
total_claim_amount vs injury_claim, property_claim, vehicle_claim
# Dropping age, incident_date and total_claim_amount as they do not show any
statistical significance for predicting
X_train_resampled.drop(['age', 'total_claim_amount', 'incident_date'],
axis=1, inplace=True)
X_test.drop(['age', 'total_claim_amount', 'incident_date'], axis=1,
inplace=True)

# Check the data
display(X_train_resampled.head)
display(X_test.head())
print(X_train_resampled.info())
print(X_test.info())
```

6.4 Combine values in Categorical Columns [6 Marks]

During the EDA process, categorical columns with multiple unique values may be identified. To enhance model performance, it is essential to refine these categorical features by grouping values that have low frequency or provide limited predictive information.

Combine categories that occur infrequently or exhibit similar behavior to reduce sparsity and improve model generalisation.

```
# Combine categories that have low frequency or provide limited predictive
information

# Define bins and labels and create a new category for
incident_hour_of_the_day
bins = [0, 6, 12, 18, 24]
labels = ['Night', 'Morning', 'Afternoon', 'Evening']

#Combine categories for Training data
X_train_resampled['incident_period_of_the_day'] =
pd.cut(X_train_resampled['incident_hour_of_the_day'], bins=bins,
labels=labels, right=False)

#Combine categories for validation data
X_test['incident_period_of_the_day'] =
pd.cut(X_test['incident_hour_of_the_day'], bins=bins, labels=labels,
right=False)

#Drop the feature incident_hour_of_the_day for training and validation data
X_train_resampled.drop('incident_hour_of_the_day', axis=1, inplace=True)
X_test.drop('incident_hour_of_the_day', axis=1, inplace=True)
```

6.5 Dummy variable creation [6 Marks]

Transform categorical variables into numerical representations using dummy variables. Ensure consistent encoding between training and validation data.

6.5.1 Identify categorical columns for dummy variable creation [1 Mark]

```
# Identify the categorical columns for creating dummy variables
categorical_cols =
X_train_resampled.select_dtypes(exclude=['int64','int32','float64']).columns.
tolist()
categorical_cols
```

```
['policy_state',
 'policy_csl',
 'insured_sex',
 'insured_education_level',
 'insured_occupation',
 'insured_hobbies',
 'insured_relationship',
 'incident_type',
 'collision_type',
 'incident_severity',
 'authorities_contacted',
 'incident_state',
 'incident_city',
 'property_damage',
```

```
 'police_report_available',
 'auto_make',
 'auto_model',
 'incident_period_of_the_day']
```

***6.5.2** Create dummy variables for categorical columns in training data [2 Marks]*

```
# Create dummy variables using the 'get_dummies' for categorical columns in
training data
X_train_resampled = pd.get_dummies(X_train_resampled,
columns=categorical_cols, dtype=int, drop_first=True)
X_train_resampled
```

{"type":"dataframe","variable_name":"X_train_resampled"}

```
X_test
```

{"type":"dataframe","variable_name":"X_test"}

***6.5.3** Create dummy variables for categorical columns in validation data [2 Marks]*

```
# Create dummy variables using the 'get_dummies' for categorical columns in
validation data
X_test = pd.get_dummies(X_test, columns=categorical_cols, dtype=int,
drop_first=True)
X_test
```

{"type":"dataframe","variable_name":"X_test"}

***6.5.4** Create dummy variable for dependent feature in training and validation data [1 Mark]*

```
# The dependent feature 'fraud_reported' is already encoded as 0 and 1, so no
dummy variable creation is needed.
```

6.6 Feature scaling [3 marks]

Scale numerical features to a common range to prevent features with larger values from dominating the model. Choose a scaling method appropriate for the data and the chosen model. Apply the same scaling to both training and validation data.

```
X_train_resampled.columns.tolist()
```

```
# Import the necessary scaling tool from scikit-learn
from sklearn.preprocessing import StandardScaler

# Scale the numeric features present in the training data
scaler = StandardScaler()
features_to_scale = ['months_as_customer', 'policy_deductable',
'policy_annual_premium','umbrella_limit', 'capital-gains', 'capital-
loss','number_of_vehicles_involved', 'bodily_injuries',
'witnesses','injury_claim', 'property_claim', 'vehicle_claim',
'incident_month', 'auto_age']
X_train_resampled[features_to_scale] =
scaler.fit_transform(X_train_resampled[features_to_scale])
```

```
# Scale the numeric features present in the validation data
X_test[features_to_scale] = scaler.transform(X_test[features_to_scale])
```

```
X_train_resampled
```

{"type":"dataframe","variable_name":"X_train_resampled"}

7. Model Building [50 marks]

In this task, you will be building two machine learning models: Logistic Regression and Random Forest. Each model will go through a structured process to ensure optimal performance. The key steps for each model are outlined below:

Logistic Regression Model

- Feature Selection using RFECV – Identify the most relevant features using Recursive Feature Elimination with Cross-Validation.
- Model Building and Multicollinearity Assessment – Build the logistic regression model and analyse statistical aspects such as p-values and VIFs to detect multicollinearity.
- Model Training and Evaluation on Training Data – Fit the model on the training data and assess initial performance.
- Finding the Optimal Cutoff – Determine the best probability threshold by analysing the sensitivity-specificity tradeoff and precision-recall tradeoff.
- FInal Prediction and Evaluation on Training Data using the Optimal Cutoff – Generate final predictions using the selected cutoff and evaluate model performance.

Random Forest Model

- Get Feature Importances - Obtain the importance scores for each feature and select the important features to train the model.
- Model Evaluation on Training Data – Assess performance metrics on the training data.
- Check Model Overfitting using Cross-Validation – Evaluate generalisation by performing cross-validation.
- Hyperparameter Tuning using Grid Search – Optimise model performance by fine-tuning hyperparameters.
- Final Model and Evaluation on Training Data – Train the final model using the best parameters and assess its performance.

7.1 Feature selection [4 marks]

Identify and select the most relevant features for building a logistic regression model using Recursive Feature Elimination with Cross-Validation (RFECV).

7.1.1 Import necessary libraries [1 Mark]

```
# Import necessary libraries
from sklearn.model_selection import StratifiedKFold
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
```

7.1.2 Perform feature selection [2 Mark]

```
# Apply RFECV to identify the most relevant features
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rfecv = RFECV(estimator=LogisticRegression(random_state=42), step=1, cv=5,
scoring='f1')
rfecv.fit(X_train_resampled, y_train_resampled)
```

```
RFECV(cv=5, estimator=LogisticRegression(random_state=42), scoring='f1')
```

```
# Display the features ranking by RFECV in a DataFrame
ranking_df = pd.DataFrame({'Feature': X_train_resampled.columns, 'Ranking':
rfecv.ranking_, "Selected": rfecv.support_})
print(f"Optimal number of features: {rfecv.n_features_}")
display(ranking_df.sort_values(by='Ranking'))
```

```
Optimal number of features: 102
```

7.1.2 Retain the selected features [1 Mark]

```
# Put columns selected by RFECV into variable 'col'
col = list(ranking_df[ranking_df['Ranking'] == 1]['Feature'])
col
```

7.2 Build Logistic Regression Model [12 marks]

After selecting the optimal features using RFECV, utilise these features to build a logistic regression model with Statsmodels. This approach enables a detailed statistical analysis of the model, including the assessment of p-values and Variance Inflation Factors (VIFs). Evaluating these metrics is crucial for detecting multicollinearity and ensuring that the selected predictors are not highly correlated.

7.2.1 Select relevant features and add constant in training data [1 Mark]

```
# Select only the columns selected by RFECV
X_train_resampled = X_train_resampled[col]
```

```
# Import statsmodels and add constant
import statsmodels.api as sm
X_train_resampled_sm = sm.add_constant(X_train_resampled)

# Check the data
print(X_train_resampled_sm)
```

7.2.2 Fit logistic regression model [2 Marks]

```
# Fit a logistic Regression model on X_train after adding a constant and
output the summary
logm1 = sm.GLM(y_train_resampled, X_train_resampled_sm,
```

```
family=sm.families.Binomial())
res = logm1.fit()
print(res.summary())
```

Model Interpretation

The output summary table will provide the features used for building model along with coefficient of each of the feature and their p-value. The p-value in a logistic regression model is used to assess the statistical significance of each coefficient. Lesser the p-value, more significant the feature is in the model.

A positive coefficient will indicate that an increase in the value of feature would increase the odds of the event occurring. On the other hand, a negative coefficient means the opposite, i.e, an increase in the value of feature would decrease the odds of the event occurring.

Now check VIFs for presence of multicollinearity in the model.

7.2.3 *Evaluate VIF of features to assess multicollinearity [2 Marks]*

```
# Import 'variance_inflation_factor'
from statsmodels.stats.outliers_influence import variance_inflation_factor

# Make a VIF DataFrame for all the variables present
vif = pd.DataFrame()
vif['Features'] = X_train_resampled_sm.columns
vif['VIF'] = [variance_inflation_factor(X_train_resampled_sm.values, i) for i
in range(X_train_resampled_sm.shape[1])]
vif = vif.sort_values(by = "VIF", ascending = False)
print(vif)
```

```
                      Features       VIF
93               auto_model_RAM       inf
97            auto_model_Ultima       inf
79         auto_model_Forrestor       inf
67           auto_make_Mercedes       inf
68             auto_make_Nissan       inf
..                          ...       ...
96                auto_model_TL  1.211911
3                     witnesses  1.204283
1                umbrella_limit  1.202164
23  insured_hobbies_basketball  1.196813
2                  capital-loss  1.184244

[103 rows x 2 columns]
```

```
vif[vif['VIF'] > 5]
```

{"repr_error":"Out of range float values are not JSON compliant: inf","type":"dataframe"}

```
high_vif_features = ['auto_model_RAM', 'auto_model_Ultima',
'auto_model_Forrestor', 'auto_make_Mercedes', 'auto_make_Nissan',
'auto_make_Suburu', 'auto_make_Dodge', 'auto_model_Pathfinder',
'auto_model_C300', 'auto_model_Impreza', 'auto_model_E400',
'auto_model_Legacy', 'auto_model_Neon', 'auto_model_Maxima',
'auto_model_ML350', 'auto_make_BMW']
high_vif_features
```

```
['auto_model_RAM',
 'auto_model_Ultima',
 'auto_model_Forrestor',
 'auto_make_Mercedes',
 'auto_make_Nissan',
 'auto_make_Suburu',
 'auto_make_Dodge',
 'auto_model_Pathfinder',
 'auto_model_C300',
 'auto_model_Impreza',
 'auto_model_E400',
 'auto_model_Legacy',
 'auto_model_Neon',
 'auto_model_Maxima',
 'auto_model_ML350',
 'auto_make_BMW']
```

```
#drop high_vif_features
X_train_resampled_sm.drop(high_vif_features, axis=1, inplace=True)
```

```
# Fit a logistic Regression model on X_train after adding a constant and
output the summary
logm_2 = sm.GLM(y_train_resampled, X_train_resampled_sm, family =
sm.families.Binomial())
result2 = logm_2.fit()
result2.summary()
```

```
# Make a VIF DataFrame for all the variables present
vif = pd.DataFrame()
vif['Features'] = X_train_resampled_sm.columns
vif['VIF'] = [variance_inflation_factor(X_train_resampled_sm.values, i) for i
in range(X_train_resampled_sm.shape[1])]
vif = vif.sort_values(by = "VIF", ascending = False)
print(vif)
```

```
               Features        VIF
0                 const  39.776955
64      auto_make_Honda   4.183489
65     auto_make_Toyota   3.352260
68     auto_model_Civic   3.149675
63  auto_make_Chevrolet   2.868143
..                  ...        ...
82  auto_model_Wrangler   1.154986
```

```
76       auto_model_MDX   1.153928
78    auto_model_Passat   1.146613
84        auto_model_X6   1.129802
81        auto_model_TL   1.114989

[87 rows x 2 columns]
```

```
vif[vif['VIF'] > 5]
```

Proceed to the next step if p-values and VIFs are within acceptable ranges. If you observe high p-values or VIFs, drop the features and retrain the model. [THIS IS OPTIONAL]

7.2.4 *Make predictions on training data [1 Mark]*

```
# Predict the probabilities on the training data
y_train_pred = result2.predict(X_train_resampled_sm)

# Reshape it into an array
y_train_pred = y_train_pred.values.reshape(-1)
y_train_pred[:20]
```

```
array([1.50949053e-02, 1.02354473e-02, 1.48002383e-04, 9.85862765e-03,
       1.04732760e-01, 3.62300517e-02, 7.89403281e-02, 1.23827611e-03,
       1.95066709e-02, 9.77356882e-01, 9.30556127e-02, 9.56257429e-01,
       1.13698013e-11, 7.09530152e-01, 3.23566509e-03, 9.44297146e-01,
       6.36340517e-01, 4.48567879e-02, 7.13580957e-01, 9.98454184e-01])
```

7.2.5 *Create a DataFrame that includes actual fraud reported flags, predicted probabilities, and a column indicating predicted classifications based on a cutoff value of 0.5 [1 Mark]*

```
# Create a new DataFrame containing the actual fraud reported flag and the
probabilities predicted by the model
y_train_pred_df = pd.DataFrame({'Fraud_Reported': y_train_resampled.values,
'Prob': y_train_pred})

# Create new column indicating predicted classifications based on a cutoff
value of 0.5
y_train_pred_df['Predicted'] = y_train_pred_df['Prob'].apply(lambda x: 1 if x
> 0.5 else 0)
```

Model performance evaluation

Evaluate the performance of the model based on predictions made on the training data.

7.2.6 *Check the accuracy of the model [1 Mark]*

```
# Import metrics from sklearn for evaluation
from sklearn import metrics

# Check the accuracy of the model
print(metrics.accuracy_score(y_train_pred_df['Fraud_Reported'],
y_train_pred_df['Predicted']))
```

```
0.8994307400379506
```

7.2.7 *Create a confusion matrix based on the predictions made on the training data [1 Mark]*

```
# Create confusion matrix
confusion_matrix =
metrics.confusion_matrix(y_train_pred_df['Fraud_Reported'],
y_train_pred_df['Predicted'])
print(confusion_matrix)
```

```
[[467  60]
 [ 46 481]]
```

7.2.8 *Create variables for true positive, true negative, false positive and false negative [1 Mark]*

```
# Create variables for true positive, true negative, false positive and false
negative
TN = confusion_matrix[0, 0]
FP = confusion_matrix[0, 1]
FN = confusion_matrix[1, 0]
TP = confusion_matrix[1, 1]
```

7.2.9 *Calculate sensitivity, specificity, precision, recall and F1-score [2 Marks]*

```
# Calculate the sensitivity
sensitivity = TP / (TP + FN)
print(f"Sensitivity: {sensitivity}")

# Calculate the specificity
specificity = TN / (TN + FP)
print(f"Specificity: {specificity}")

# Calculate Precision
precision = TP / (TP + FP)
print(f"Precision: {precision}")

# Calculate Recall
recall = TP / (TP + FN)
print(f"Recall: {recall}")

# Calculate F1 Score
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"F1 Score: {f1_score}")
```

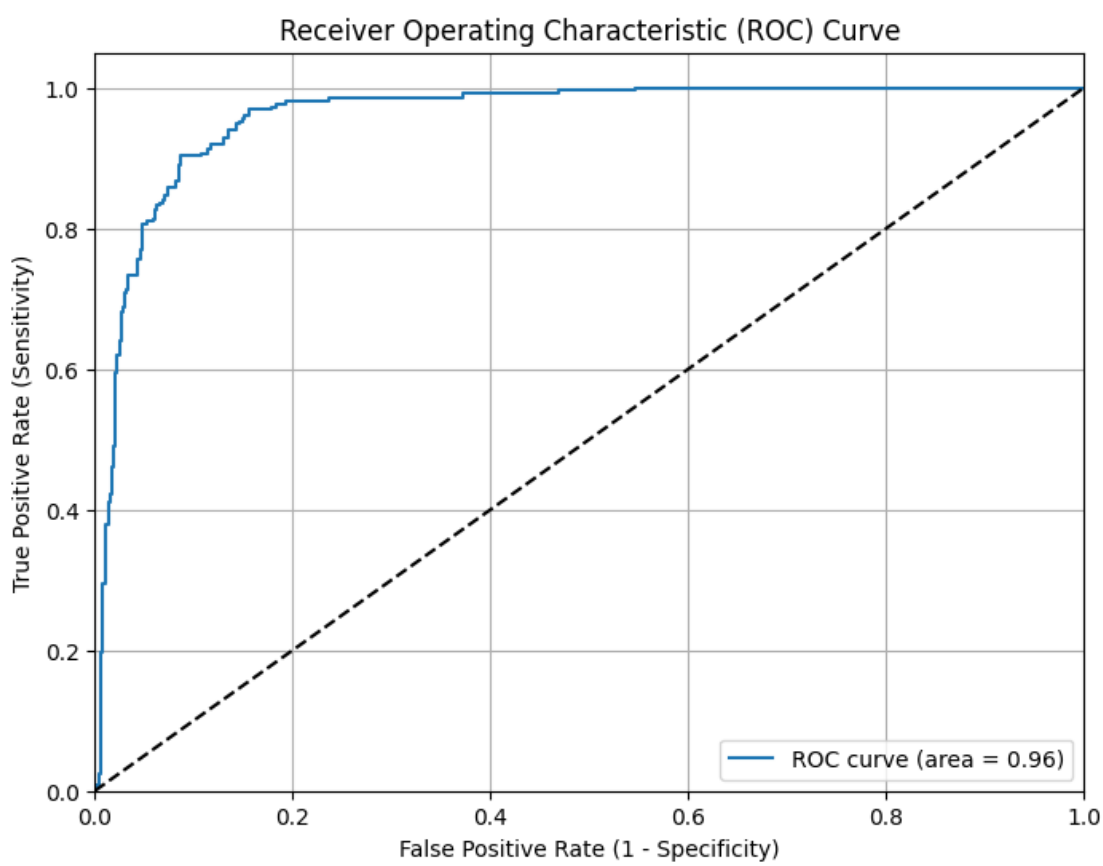
```
Sensitivity: 0.9127134724857685
Specificity: 0.8861480075901328
Precision: 0.88909426987061
Recall: 0.9127134724857685
F1 Score: 0.9007490636704121
```

7.3 Find the Optimal Cutoff [12 marks]

Find the optimal cutoff to improve model performance by evaluating various cutoff values and their impact on relevant metrics.

***7.3.1** Plot ROC Curve to visualise the trade-off between true positive rate and false positive rate across different classification thresholds [2 Marks]*

```
# Import libraries or function to plot the ROC curve
from sklearn.metrics import roc_curve, roc_auc_score

# Define ROC function
def plot_roc(actual, probs):
    fpr, tpr, thresholds = roc_curve(actual, probs, drop_intermediate=False)
    auc_score = roc_auc_score(actual, probs)
    plt.figure(figsize=(8, 6))
    plt.plot(fpr, tpr, label=f'ROC curve (area = {auc_score:.2f})')
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate (1 - Specificity)')
    plt.ylabel('True Positive Rate (Sensitivity)')
    plt.title('Receiver Operating Characteristic (ROC) Curve')
    plt.legend(loc="lower right")
    plt.grid(True)
    plt.show()

# Call the ROC function
plot_roc(y_train_pred_df['Fraud_Reported'], y_train_pred_df['Prob'])
```



Sensitivity and Specificity tradeoff

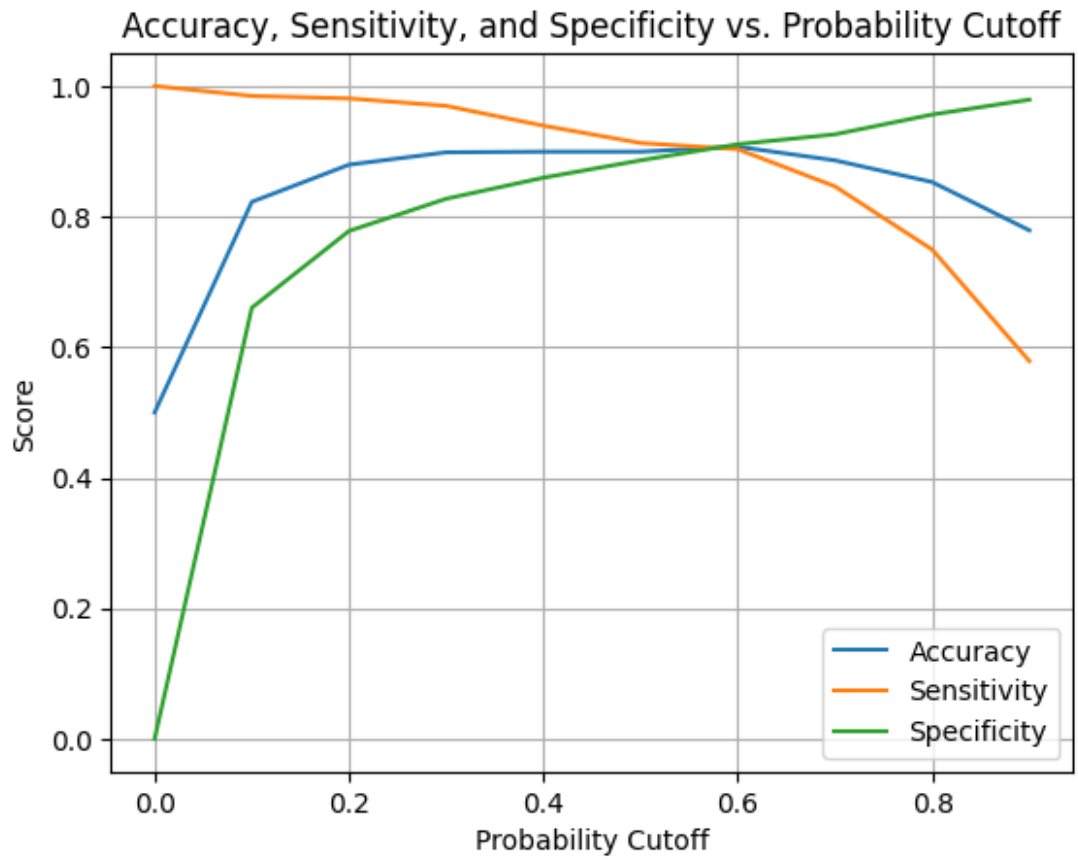
After analysing the area under the curve of the ROC, check the sensitivity and specificity tradeoff to find the optimal cutoff point.

7.3.2 *Predict on training data at various probability cutoffs [1 Mark]*

```
# Create columns with different probability cutoffs to explore the impact of
cutoff on model performance

numbers = [float(x)/10 for x in range(10)]
for i in numbers:
    y_train_pred_df[f'Predicted_{i}'] = y_train_pred_df['Prob'].apply(lambda
x: 1 if x > i else 0)
```

7.3.3 *Plot accuracy, sensitivity, specificity at different values of probability cutoffs [2 Marks]*

```
# Create a DataFrame to see the values of accuracy, sensitivity, and
specificity at different values of probability cutoffs

cutoff_df = pd.DataFrame(columns=['Prob', 'Accuracy', 'Sensitivity',
'Specificity'])

for i in numbers:
    cm = metrics.confusion_matrix(y_train_pred_df['Fraud_Reported'],
y_train_pred_df[f'Predicted_{i}'])
    total = sum(sum(cm))
    accuracy = (cm[0,0] + cm[1,1]) / total
    specificity = cm[0,0] / (cm[0,1] + cm[0,0])
    sensitivity = cm[1,1] / (cm[1,0] + cm[1,1])
    cutoff_df.loc[i] = [i, accuracy, sensitivity, specificity]

display(cutoff_df)

cutoff_df.plot.line(x='Prob', y=['Accuracy', 'Sensitivity', 'Specificity'])
plt.title('Accuracy, Sensitivity, and Specificity vs. Probability Cutoff')
plt.xlabel('Probability Cutoff')
plt.ylabel('Score')
plt.grid(True)
plt.show()
```



***7.3.4** Create a column for final prediction based on optimal cutoff [1 Mark]*

```
# Create a column for final prediction based on the optimal cutoff
y_train_pred_df['final_predicted'] = y_train_pred_df['Prob'].apply(lambda x:
1 if x > 0.5 else 0)
```

***7.3.5** Calculate the accuracy [1 Mark]*

```
# Check the accuracy now
print(metrics.accuracy_score(y_train_pred_df['Fraud_Reported'],
y_train_pred_df['final_predicted']))
```

```
0.8994307400379506
```

***7.3.6** Create confusion matrix [1 Mark]*

```
# Create the confusion matrix once again
confusion_matrix =
metrics.confusion_matrix(y_train_pred_df['Fraud_Reported'],
y_train_pred_df['final_predicted'])
print(confusion_matrix)
```

```
[[467  60]
 [ 46 481]]
```

7.3.7 *Create variables for true positive, true negative, false positive and false negative [1 Mark]*

```
# Create variables for true positive, true negative, false positive and false negative
TN = confusion_matrix[0, 0]
FP = confusion_matrix[0, 1]
FN = confusion_matrix[1, 0]
TP = confusion_matrix[1, 1]
```

7.3.8 *Calculate sensitivity, specificity, precision, recall and F1-score of the model [2 Mark]*

```
# Calculate the sensitivity
sensitivity = TP / (TP + FN)
print(f"Sensitivity: {sensitivity}")

# Calculate the specificity
specificity = TN / (TN + FP)
print(f"Specificity: {specificity}")

# Calculate Precision
precision = TP / (TP + FP)
print(f"Precision: {precision}")

# Calculate Recall
recall = TP / (TP + FN)
print(f"Recall: {recall}")

# Calculate F1 Score
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"F1 Score: {f1_score}")
```

```
Sensitivity: 0.9127134724857685
Specificity: 0.8861480075901328
Precision: 0.88909426987061
Recall: 0.9127134724857685
F1 Score: 0.9007490636704121
```

Precision and Recall tradeoff

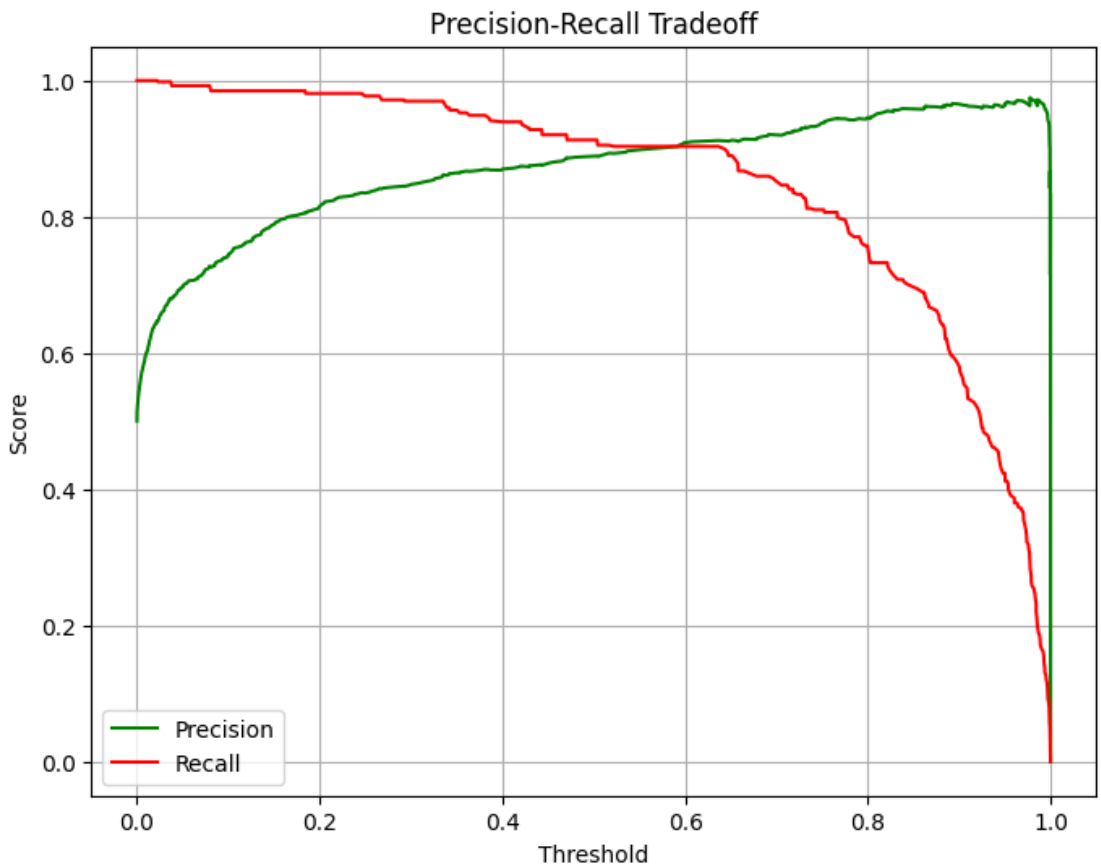
Check optimal cutoff value by plotting precision-recall curve, and adjust the cutoff based on precision and recall tradeoff if required.

```
# Import precision-recall curve function
from sklearn.metrics import precision_recall_curve
```

7.3.9 *Plot precision-recall curve [1 Mark]*

```
# Plot precision-recall curve
precision, recall, thresholds =
precision_recall_curve(y_train_pred_df['Fraud_Reported'],
y_train_pred_df['Prob'])
plt.figure(figsize=(8, 6))
plt.plot(thresholds, precision[:-1], "g-", label="Precision")
```

```
plt.plot(thresholds, recall[:-1], "r-", label="Recall")
plt.title('Precision-Recall Tradeoff')
plt.xlabel('Threshold')
plt.ylabel('Score')
plt.legend(loc="lower left")
plt.grid(True)
plt.show()
```



7.4 Build Random Forest Model [12 marks]

Now that you have built a logistic regression model, let's move on to building a random forest model.

7.4.1 Import necessary libraries

```
# Import necessary libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
from sklearn.model_selection import cross_val_score, GridSearchCV
```

7.4.2 Build the random forest model [1 Mark]

```
# Build a base random forest model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train_resampled, y_train_resampled)
```

```
RandomForestClassifier(random_state=42)
```

7.4.3 *Get feature importance scores and select important features [2 Marks]*

```
# Get feature importance scores from the trained model
feature_importances = pd.DataFrame({'feature': X_train_resampled.columns,
'importance': rf_model.feature_importances_})

# Create a DataFrame to visualise the importance scores
feature_importances = feature_importances.sort_values(by='importance',
ascending=False)
display(feature_importances)

# Select features with high importance scores
selected_features_rf =
feature_importances[feature_importances['importance']>=0.02]['feature'].tolis
t()
print(selected_features_rf)
print(f"Number of selected important features: {len(selected_features_rf)}")
# Create a new training data with only the selected features
X_train_resampled_rf = X_train_resampled[selected_features_rf]
X_train_resampled_rf.shape
```

```
['incident_severity_Minor Damage', 'vehicle_claim', 'insured_hobbies_chess',
'incident_severity_Total Loss', 'capital-loss', 'insured_hobbies_cross-fit',
'witnesses']
Number of selected important features: 7
```

```
(1054, 7)
```

7.4.4 *Train the model with selected features [1 Mark]*

```
# Fit the model on the training data with selected features
rf_model.fit(X_train_resampled_rf, y_train_resampled)
```

```
RandomForestClassifier(random_state=42)
```

7.4.5 *Generate predictions on the training data [1 Mark]*

```
# Generate predictions on training data
y_train_pred_rf = rf_model.predict(X_train_resampled_rf)
```

7.4.6 *Check accuracy of the model [1 Mark]*

```
# Check accuracy of the model
print(metrics.accuracy_score(y_train_resampled, y_train_pred_rf))
```

```
1.0
```

7.4.7 *Create confusion matrix [1 Mark]*

```
# Create the confusion matrix to visualise the performance
confusion_matrix_rf = metrics.confusion_matrix(y_train_resampled,
y_train_pred_rf)
print(confusion_matrix_rf)
```

```
[[527   0]
 [  0 527]]
```

7.4.8 *Create variables for true positive, true negative, false positive and false negative [1 Mark]*

```
# Create variables for true positive, true negative, false positive and false negative
TN = confusion_matrix_rf[0, 0]
FP = confusion_matrix_rf[0, 1]
FN = confusion_matrix_rf[1, 0]
TP = confusion_matrix_rf[1, 1]
```

7.4.9 *Calculate sensitivity, specificity, precision, recall and F1-score of the model [2 Marks]*

```
# Calculate the sensitivity
sensitivity = TP / (TP + FN)
print(f"Sensitivity: {sensitivity}")

# Calculate the specificity
specificity = TN / (TN + FP)
print(f"Specificity: {specificity}")

# Calculate Precision
precision = TP / (TP + FP)
print(f"Precision: {precision}")

# Calculate Recall
recall = TP / (TP + FN)
print(f"Recall: {recall}")

# Calculate F1 Score
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"F1 Score: {f1_score}")
```

```
Sensitivity: 1.0
Specificity: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0
```

7.4.10 *Check if the model is overfitting training data using cross validation [2 marks]*

```
# Use cross validation to check if the model is overfitting
cv_scores = cross_val_score(rf_model, X_train_resampled_rf,
y_train_resampled, cv=5, scoring='accuracy')
print(f"Cross-validation accuracy scores: {cv_scores}")
print(f"Mean cross-validation accuracy: {cv_scores.mean()}")
print(f"Standard deviation of cross-validation accuracy: {cv_scores.std()}")
```

```
Cross-validation accuracy scores: [0.91943128 0.90521327 0.92890995
0.91469194 0.93809524]
Mean cross-validation accuracy: 0.9212683367185737
Standard deviation of cross-validation accuracy: 0.011365982557896316
```

7.5 Hyperparameter Tuning [10 Marks]

Enhance the performance of the random forest model by systematically exploring and selecting optimal hyperparameter values using grid search.

7.5.1 Use grid search to find the best hyperparameter values [2 Marks]

```
# Use grid search to find the best hyperparamter values
param_grid = {
    'n_estimators': [10,15,20,25,30,35,40],    # Keep it modest; 50–100 trees
is enough
    'max_depth': [5,10,15,20],                 # Restrict depth to avoid
deep, overfitting trees
    'min_samples_split': [20,25,30],           # Require more samples to
split; reduces overfitting
    'min_samples_leaf': [10,15,20],            # Larger leaves generalize
better
    'max_features': [8,12,16]                  # Try sqrt (default) or a
fixed number of features
}

grid_search = GridSearchCV(estimator=RandomForestClassifier(random_state=42),
param_grid=param_grid, cv=5, n_jobs=-1, verbose=2, scoring='accuracy')
grid_search.fit(X_train_resampled_rf, y_train_resampled)

# Best Hyperparameters
best_params = grid_search.best_params_
print(f"Best Hyperparameters: {best_params}")
```

```
Fitting 5 folds for each of 756 candidates, totalling 3780 fits
Best Hyperparameters: {'max_depth': 10, 'max_features': 8,
'min_samples_leaf': 10, 'min_samples_split': 20, 'n_estimators': 15}
```

7.5.2 Build a random forest model based on hyperparameter tuning results [1 Mark]

```
# Building random forest model based on results of hyperparameter tuning
rf_tuned_model = RandomForestClassifier(random_state=42, **best_params)
rf_tuned_model.fit(X_train_resampled_rf, y_train_resampled)
```

```
RandomForestClassifier(max_depth=10, max_features=8, min_samples_leaf=10,
                       min_samples_split=20, n_estimators=15,
random_state=42)
```

7.5.3 Make predictions on training data [1 Mark]

```
# Make predictions on training data
y_train_pred_rf_tuned = rf_tuned_model.predict(X_train_resampled_rf)
```

7.5.4 Check accuracy of Random Forest Model [1 Mark]

```
# Check the accuracy
print(metrics.accuracy_score(y_train_resampled, y_train_pred_rf_tuned))
```

```
0.8624288425047438
```

7.5.5 *Create confusion matrix [1 Mark]*

```
# Create the confusion matrix
confusion_matrix_rf_tuned = metrics.confusion_matrix(y_train_resampled,
y_train_pred_rf_tuned)
print(confusion_matrix_rf_tuned)
```

```
[[450  77]
 [ 68 459]]
```

7.5.6 *Create variables for true positive, true negative, false positive and false negative [1 Mark]*

```
# Create variables for true positive, true negative, false positive and false
negative
TN = confusion_matrix_rf_tuned[0, 0]
FP = confusion_matrix_rf_tuned[0, 1]
FN = confusion_matrix_rf_tuned[1, 0]
TP = confusion_matrix_rf_tuned[1, 1]
```

7.5.7 *Calculate sensitivity, specificity, precision, recall and F1-score of the model [3 Marks]*

```
# Calculate the sensitivity
sensitivity = TP / (TP + FN)
print(f"Sensitivity: {sensitivity}")

# Calculate the specificity
specificity = TN / (TN + FP)
print(f"Specificity: {specificity}")

# Calculate Precision
precision = TP / (TP + FP)
print(f"Precision: {precision}")

# Calculate Recall
recall = TP / (TP + FN)
print(f"Recall: {recall}")

# Calculate F1-Score
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"F1 Score: {f1_score}")
```

```
Sensitivity: 0.8709677419354839
Specificity: 0.8538899430740038
Precision: 0.8563432835820896
Recall: 0.8709677419354839
F1 Score: 0.863593603010348
```

8. Prediction and Model Evaluation [20 marks]

Use the model from the previous step to make predictions on the validation data with the optimal cutoff. Then evaluate the model's performance using metrics such as accuracy, sensitivity, specificity, precision, and recall.

8.1 Make predictions over validation data using logistic regression model [10 marks]

8.1.1 *Select relevant features for validation data and add constant [1 Mark]*

```
cols_new = X_train_resampled_sm.columns.tolist()

cols_new.remove('const')

cols_new

# Select the relevant features for validation data
X_test_lr = X_test[cols_new]

# Add constant to X_validation
X_test_sm = sm.add_constant(X_test_lr)
```

8.1.2 *Make predictions over validation data [1 Mark]*

```
# Make predictions on the validation data and store it in the variable
'y_validation_pred_proba'
y_test_pred_proba = result2.predict(X_test_sm)
```

8.1.3 *Create DataFrame with actual values and predicted values for validation data [2 Marks]*

```
#  Create DataFrame with actual values and predicted values for validation
data
y_test_pred_df = pd.DataFrame({'Fraud_Reported': y_test.values, 'Prob':
y_test_pred_proba})
```

8.1.4 *Make final prediction based on cutoff value [1 Mark]*

```
# Make final predictions on the validation data using the optimal cutoff
y_test_pred_df['final_predicted'] = y_test_pred_df['Prob'].apply(lambda x: 1
if x > 0.56 else 0)
```

8.1.5 *Check the accuracy of logistic regression model on validation data [1 Mark]*

```
# Check the accuracy
print(metrics.accuracy_score(y_test_pred_df['Fraud_Reported'],
y_test_pred_df['final_predicted']))
```

```
0.8033333333333333
```

8.1.6 *Create confusion matrix [1 Mark]*

```
# Create the confusion matrix
confusion_matrix = metrics.confusion_matrix(y_test_pred_df['Fraud_Reported'],
y_test_pred_df['final_predicted'])
print(confusion_matrix)
```

```
[[195  31]
 [ 28  46]]
```

8.1.7 *Create variables for true positive, true negative, false positive and false negative [1 Mark]*

```
# Create variables for true positive, true negative, false positive and false
negative
TN = confusion_matrix[0, 0]
```

```
FP = confusion_matrix[0, 1]
FN = confusion_matrix[1, 0]
TP = confusion_matrix[1, 1]
```

8.1.8 *Calculate sensitivity, specificity, precision, recall and f1 score of the model [2 Marks]*

```
# Calculate the sensitivity
sensitivity = TP / (TP + FN)
print(f"Sensitivity: {sensitivity}")

# Calculate the specificity
specificity = TN / (TN + FP)
print(f"Specificity: {specificity}")

# Calculate Precision
precision = TP / (TP + FP)
print(f"Precision: {precision}")

# Calculate Recall
recall = TP / (TP + FN)
print(f"Recall: {recall}")

# Calculate F1 Score
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"F1 Score: {f1_score}")
```

```
Sensitivity: 0.6216216216216216
Specificity: 0.8628318584070797
Precision: 0.5974025974025974
Recall: 0.6216216216216216
F1 Score: 0.6092715231788081
```

8.2 Make predictions over validation data using random forest model [10 marks]

8.2.1 *Select the important features and make predictions over validation data [2 Marks]*

```
# Select the relevant features for validation data
X_test_rf = X_test[selected_features_rf]

# Make predictions on the validation data
y_test_pred_rf = rf_tuned_model.predict(X_test_rf)
```

8.2.2 *Check accuracy of random forest model [1 Mark]*

```
# Check accuracy
print(metrics.accuracy_score(y_test, y_test_pred_rf))
```

```
0.82
```

8.2.3 *Create confusion matrix [1 Mark]*

```
# Create the confusion matrix
confusion_matrix_rf_test = metrics.confusion_matrix(y_test, y_test_pred_rf)
print(confusion_matrix_rf_test)
```

```
[[190  36]
 [ 18  56]]
```

8.2.4 *Create variables for true positive, true negative, false positive and false negative [1 Mark]*

```
# Create variables for true positive, true negative, false positive and false negative
TN = confusion_matrix_rf_test[0, 0]
FP = confusion_matrix_rf_test[0, 1]
FN = confusion_matrix_rf_test[1, 0]
TP = confusion_matrix_rf_test[1, 1]
```

8.2.5 *Calculate sensitivity, specificity, precision, recall and F1-score of the model [5 Marks]*

```
# Calculate the sensitivity
sensitivity = TP / (TP + FN)
print(f"Sensitivity: {sensitivity}")

# Calculate the specificity
specificity = TN / (TN + FP)
print(f"Specificity: {specificity}")

# Calculate Precision
precision = TP / (TP + FP)
print(f"Precision: {precision}")

# Calculate Recall
recall = TP / (TP + FN)
print(f"Recall: {recall}")

# Calculate F1-score
f1_score = 2 * (precision * recall) / (precision + recall)
print(f"F1 Score: {f1_score}")
```

```
Sensitivity: 0.7567567567567568
Specificity: 0.8407079646017699
Precision: 0.6086956521739131
Recall: 0.7567567567567568
F1 Score: 0.674698795180723
```

Evaluation and Conclusion

This assignment successfully developed and evaluated machine learning models to detect fraudulent insurance claims, fulfilling Global Insure's business objectives. The Random Forest model, after careful feature engineering and hyperparameter tuning, emerged as the most effective solution with an F1-Score of 0.675 and an accuracy of 82.00% on unseen data. This model provides a robust, data-driven approach to identify suspicious claims earlier, thereby reducing financial losses and enhancing operational efficiency. While significant progress has been made, continuous improvement through advanced modeling techniques and real-world feedback loops will further strengthen Global Insure's fraud detection capabilities.

Q&A

1. **How can we analyse historical claim data to detect patterns that indicate fraudulent claims?**

   Historical claim data was analyzed using comparative bar plots for categorical features such as incident type, collision type, incident period of day, and education level, to visualize the proportion of fraudulent claims within each category. Box plots were used for numerical features like total claim amount and auto age, comparing their distributions between legitimate and fraudulent claims. New features like `incident_period_of_the_day` and `auto_age` were also engineered for analysis.

2. **Which features are most predictive of fraudulent behaviour?**

   The Random Forest model identified several features as highly predictive of fraudulent behavior. These include `incident_severity_Minor Damage`, `incident_severity_Total Loss`, `vehicle_claim`, specific `insured_hobbies` (e.g., `chess`, `cross-fit`), `capital-loss`, and `witnesses`.

3. **Can we predict the likelihood of fraud for an incoming claim, based on past data?**

   The likelihood of fraud for incoming claims can be predicted using the **Tuned Random Forest model**. New claim data is preprocessed (scaled, encoded, and feature-selected) and then fed into the model, which outputs a probability score indicating fraud likelihood. The Tuned Random Forest model achieved the following performance on the validation data:

   - Accuracy: 82.00%
   - F1-Score: 0.675
   - Sensitivity (Recall): 75.68%
   - Specificity: 84.07%
   - Precision: 60.87%

4. **What insights can be drawn from the model that can help in improving the fraud detection process?**

   Actionable insights include deploying the Tuned Random Forest model, focusing on the identified key predictive features for data collection and targeted investigations, implementing continuous monitoring for model performance with feedback loops, and exploring advanced machine learning techniques like Gradient Boosting Models, Anomaly Detection, or Deep Learning for future improvements.